PSI

The CUDA ClusterFinder

Kernel design · hardware limits · seven optimization steps

Khalil Daniel Ferjaoui

Paul Scherrer Institut · aare

September 2026

AARE · HYBRID PIXEL DETECTORS · CUDA CLUSTERFINDER

The kernel was rarely the bottleneck. Feeding it was.

One kernel, one thread per pixel, and seven steps to keep it fed

WHERE THIS ENDS UP · THE SHIPPED f32 BUILD AFTER ALL SEVEN STEPS · ENGINE TIMES [f32 · s4] · RECONCILED IN A7

3×3	×9.1	61,312	~1 in 10^6
CLUSTER SIZE	VS 24-THREAD CPU	FRAMES / SECOND	CLUSTER MISMATCH VS CPU

RTX 4090 (Ada, sm_89) · PCIe 4.0 ×16 · Mönch 400×400 uint16 · 3×3 clusters · 100 000 frames · Cu fluorescence, MAX IV

CONTEXT · WHAT THE CODE DOES

The algorithm, the hardware, the limits

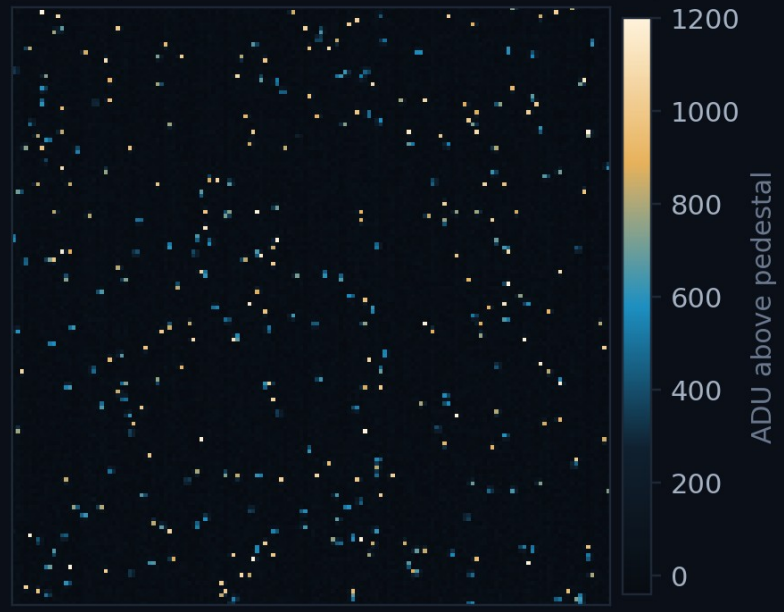
No optimizations yet, only what the hardware has to do.

COMING UP

A photon is not a pixel: it is a cluster

- One photon's charge **spreads over neighbouring pixels**; summing the 3×3 patch recovers its energy.
- The histogram of those cluster energies **is** the measurement: peak and width give gain and **energy resolution**.

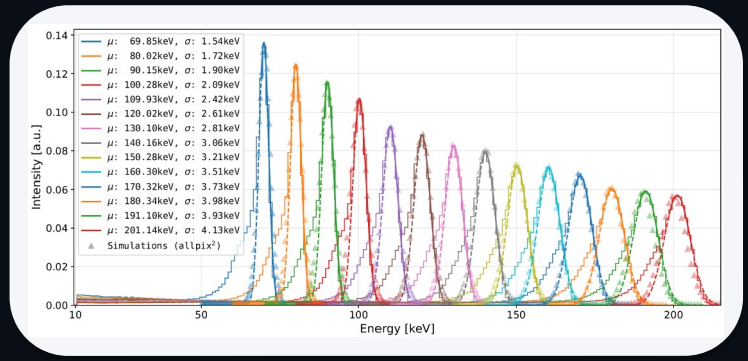
one frame, pedestal subtracted · 150×150 crop



9×9 zoom on one hit

3×3 sum = 2258 ADU — one photon.
The peak pixel holds only part of the charge.

WHY IT MATTERS



Cluster-energy spectra from an energy scan.

MÖNCH03 · THE DETECTOR THIS FEEDS

ARRAY · PITCH · ACTIVE AREA

400 × 400 · 25 µm · 10 × 10 mm²

FRAMES PER SECOND

1.3 k standard, 3–6 k optimised

PEAK PIXELS = PHOTONS / FRAME

~2 330 · 1.5 %

Real MÖNCH data, Cu fluorescence, MAX IV. One cluster per local maximum, so 2 330 counts photons.

Per pixel: subtract, threshold, update the pedestal

- Per pixel: subtract a **running pedestal**, keep what clears **nσ · rms**, cut a 3×3 cluster around each local maximum.
- The pedestal is **updated by ~80 % of pixels every frame**, so arithmetic and data movement are coupled.

THE WHOLE ALGORITHM · THREE OUTCOMES, NOT TWO

```
// the whole algorithm, per pixel
v   = frame[i] - pedestal_mean[i]
rms = pedestal rms at i
m   = max(v) over the 3x3 window at i
if (m > nSigma * rms)                  // a photon is within reach
    if (v == m) -> emit cluster        //   ... and I am its peak
    else        -> nothing             //   ... I am in its shadow
else            -> update pedestal     // I saw nothing
```

> **Thesis of this talk:** the compute was fast almost immediately. Six of the seven steps are about feeding it.

THE SHAPE OF THE WORK

WORK ITEMS PER FRAME

160 000

OPERATIONS ON EACH

~5, identical

COMMUNICATION BETWEEN THEM

none

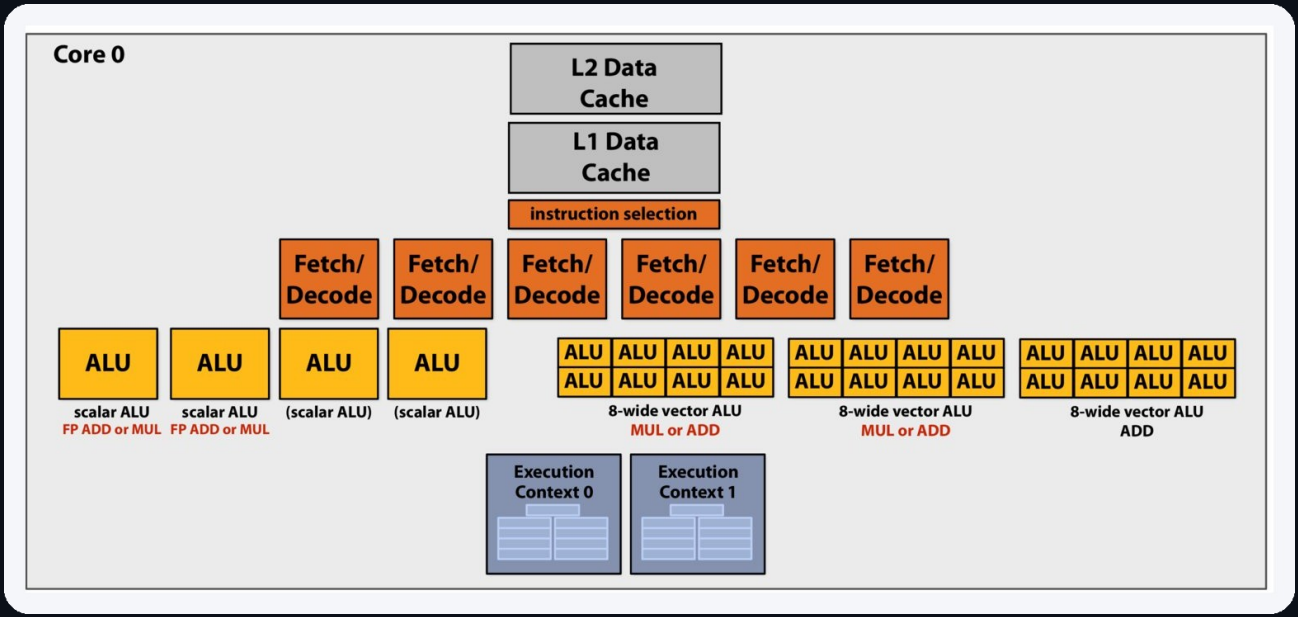
ORDER THEY MAY RUN IN

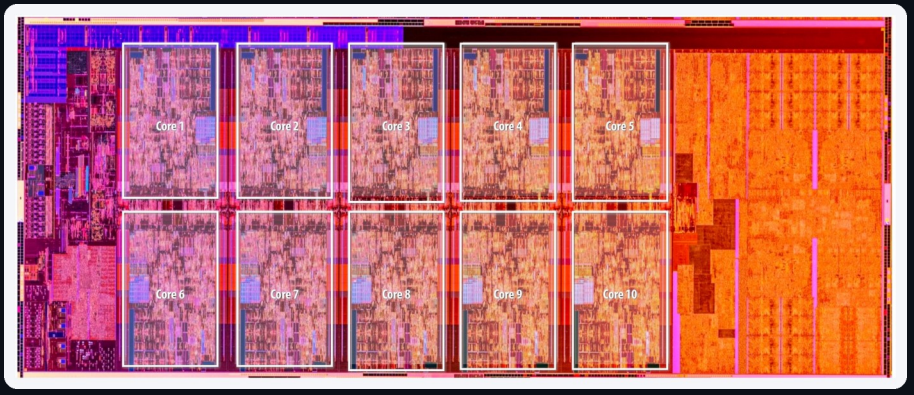
any, once the pedestal is fixed

BYTES PER FRAME

312.5 KiB · ~2 330 clusters

CPU: latency-oriented, built to finish one thread fast





Count the boxes: **6 fetch/decode** and two levels of private cache, all to keep **two** instruction streams fed. The ALUs are the small part.

Both diagrams after Stanford CS149, Fall 2025.

PC-MOENCH-04 · AMD RYZEN 9 7950X

CORES × SMT

16 × 2

CPU, 1 THREAD

1.75 ms

CPU MT, 24 THREADS

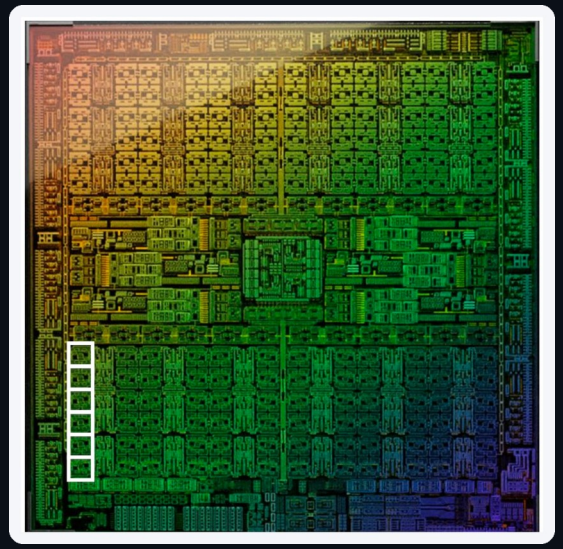
148 µs

THAT IS THE BAR

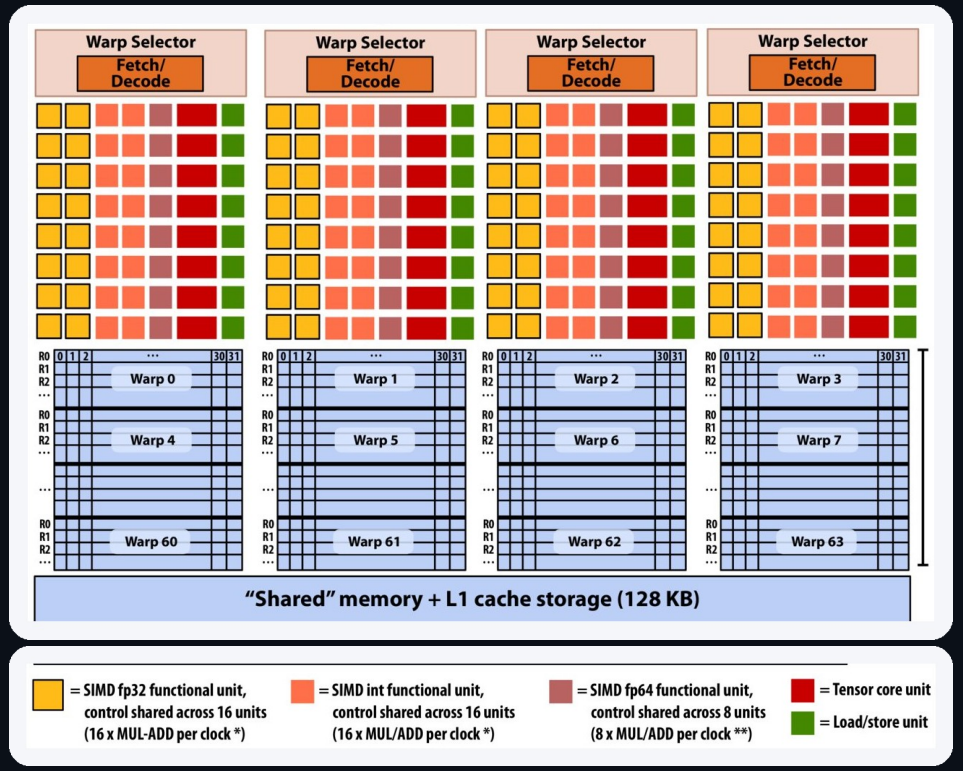
6 762 frames / s

Thread count is swept per cluster size, not assumed: 24 at 3×3, 32 at 9×9.

GPU: throughput-oriented, the whole frame at once



AD102 · 144 blocks, 128 enabled on this card. One SM boxed.



> Same grammar, inverted proportions: **4 fetch/decode** for **64 warp contexts**.

One SM: a V100 is shown; this card's is the same idea: 128 FP32 lanes, 48 warp slots, 100 kB shared memory. After Stanford CS149, Fall 2025.

NVIDIA GEFORCE RTX 4090

FP32 LANES

16 384

128 SMS × 128 LANES

vs 16 cores × 16 = 256

RESIDENT THREAD SLOTS

196 608

THE FRAME NEEDS

160 000 → all at once

Device memory runs at 1 008 GB/s (384-bit, 21 Gbps); PCIe 4.0 ×16 peaks at 31.5 and measures 24. Two copy engines, so H2D and D2H run at the same time.

One thread per pixel, one block per 16×16 tile

- A **16×16 block = 256 threads**; the grid tiles the frame in **625 identical blocks**.
- The output is **sparse**, but the **decision work is dense**: every pixel is tested, **160 000 times per frame**.

LAUNCH CONFIGURATION · ClusterFinderCUDA.hpp

```
block = dim3(BLOCK_X, BLOCK_Y);           // 16 x 16 = 256 threads
grid  = dim3((ncols + BLOCK_X - 1)/BLOCK_X,  // 25 x 25 = 625 blocks
             (nrows + BLOCK_Y - 1)/BLOCK_Y);

find_clusters_in_single_frame<<<grid, block, shmem, stream>>>(
    d_frame, d_pd_mean, /* ... 9 more ... */ d_clusters);
```

400×400 → a 25×25 grid of 16×16 blocks = 625 blocks per frame, over 128 SMs. Nothing in the launch depends on the cluster count.

WHERE THE LOOP WENT

```
// CPU: one thread per FRAME, loop inside
find_clusters(frame) {
  for (int iy = 0; iy < 400; iy++)
    for (int ix = 0; ix < 400; ix++)
      /* test pixel (iy,ix) inline */
}
// GPU: no loop. The launch is the loop:
ix = blockIdx.x*blockDim.x + threadIdx.x;
iy = blockIdx.y*blockDim.y + threadIdx.y;
```

The loop is the launch. 625 blocks × 256 threads = **160 000**, one per pixel, decided before any data is seen.

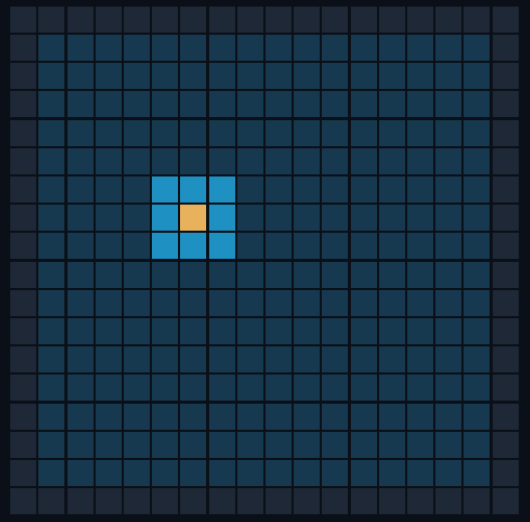
A **block** is the 256 threads that share one tile and can wait for each other. That is the next slide.

1 load tile + halo › 2 __syncthreads › 3 stencil reduction › 4 classify › 5 append or update pedestal

Shared memory: load the tile once, reuse it nine times

- Neighbouring threads need **overlapping** windows; without shared memory each pixel is fetched up to **nine times**.
- Each block stages one **pedestal-subtracted** tile plus its halo, loaded by the threads on the block edges.

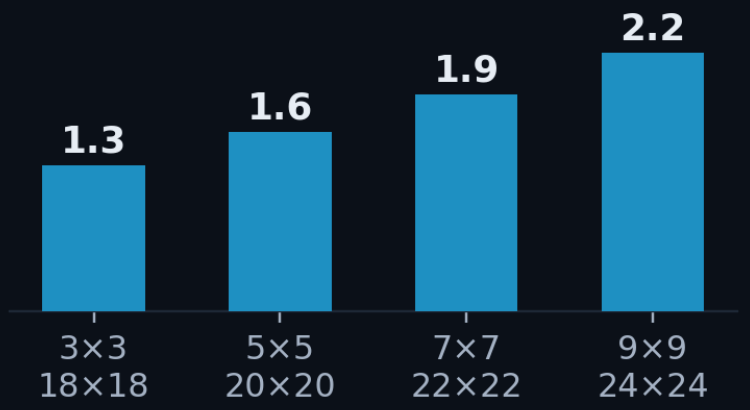
shared-memory tile · 18 × 18



the thread's own pixel
its 3×3 neighbourhood
halo · loaded, never centred on

KB of shared memory per 16×16 block (float tile)

100 KB available per SM on Ada:
shared memory is never the limit



clusterfinder_kernel.cuh

```
extern __shared__ unsigned char smem[];
auto *sh = (COMPUTE_TYPE*)smem;
auto stride = blockDim.x + 2*col_radius;
auto tid = (threadIdx.y + row_radius)*stride
         + (threadIdx.x + col_radius);
// pedestal subtraction fused into the load
sh[tid] = d_frame[gid] - d_pd_mean[gid];
```

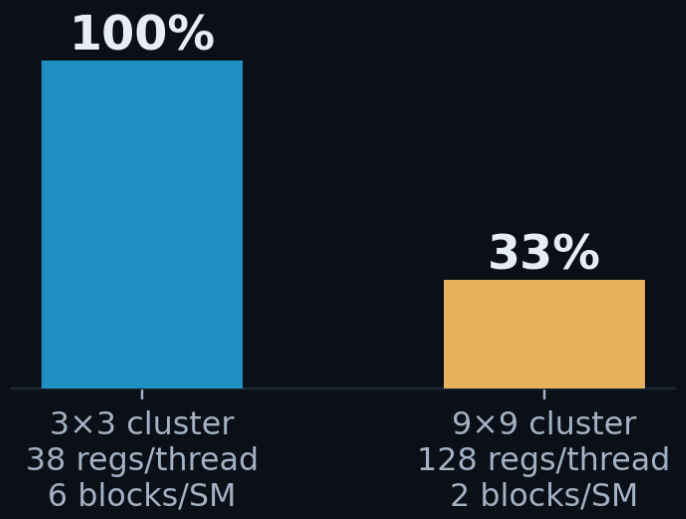
Only **odd** cluster sizes are supported, so the centre pixel is unique.

Occupancy is a latency-hiding budget

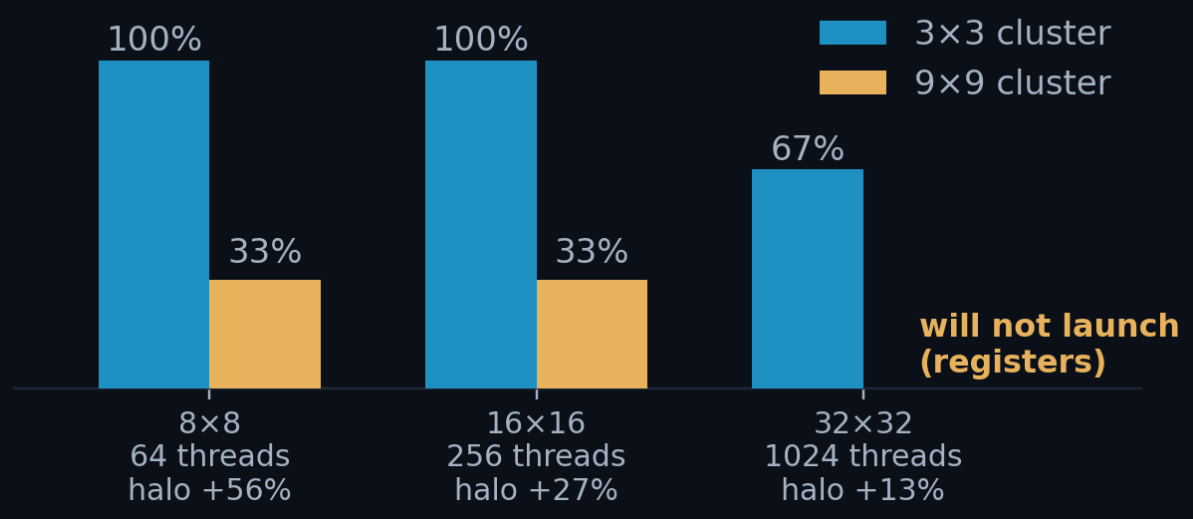
- When a warp stalls, the SM switches to another **already-resident** warp. **Occupancy = resident warps / the maximum**: how many alternatives it has to switch to.

THREADS / BLOCK	THREAD SLOTS / SM	MAX WARPS / SM	REGISTERS / SM	SHARED MEM / SM
256	1 536	48	65 536	100 kB

achieved occupancy, 16×16 block · f32 build



occupancy vs block size (halo overhead quoted for 3×3)



16×16 is the balance point. 33 % is not a failure to fix: it is what the register budget allows (**annex A1**), and at 9×9 one kernel nearly fills the machine on its own.

ROADMAP

Three acts, ordered by which bar is tallest

ACT I · FEED THE GPU

the host cannot submit work fast enough at 3×3 [f64]

opt1	**First CUDA port** 1 stream, one launch per frame	×2.34
opt2	**Streams + batching** 4 streams, drained every 4 frames	×3.66
opt3	**Pipeline rework** sync barriers removed	×4.32
opt4	**Pinned memory** DMA-speed host transfers	×5.69

ACT II · GET THE RESULTS BACK

the host copy is now the tallest bar at 3×3 [f64]

opt5	**Host↔GPU overlap** chunked submit / collect	×7.45
opt6	**Zero-copy collection** read in place, never copy	×8.65

ACT III · THE KERNEL

the kernel is the tallest bar, at 9×9 at 9×9 [f32]

opt7	**FP32 pedestal + variance rewrite** the only kernel change in the deck	kernel −41%

Speedups are 3×3 against the best CPU configuration, 24 threads.

ACT I OF III · FEED THE GPU

Bottleneck: host submission and PCIe

The kernel was fast almost immediately. This act is about the host, and it is told at 3×3, where PCIe is the floor.

STARTING FROM

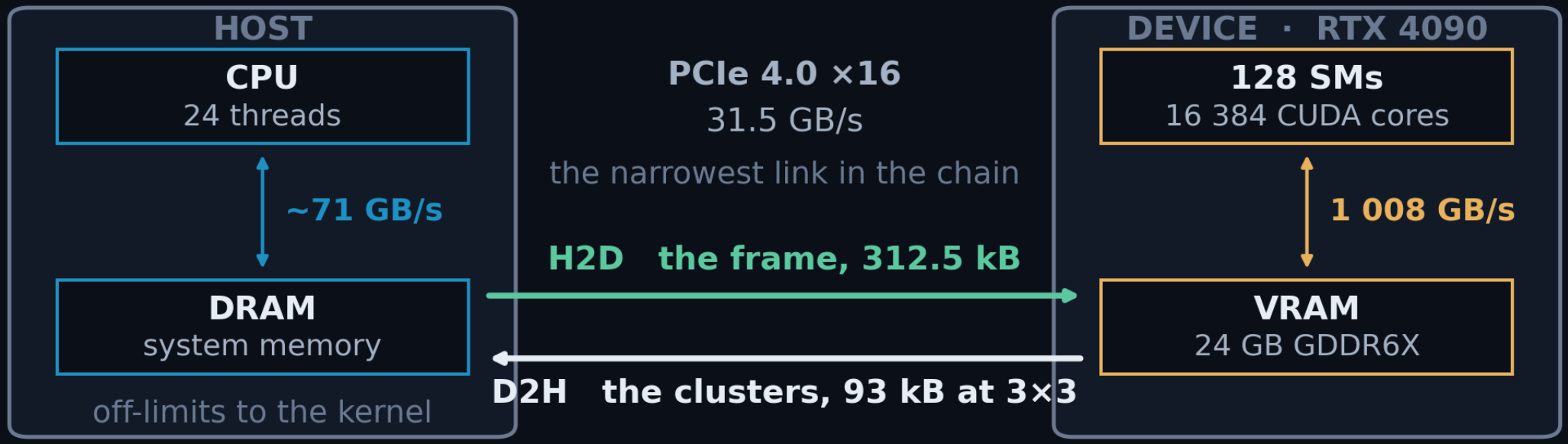
6 762 FPS

24-thread CPU · 14.8 s for 100 000 frames

COMING UP

The GPU consumes 32× faster than PCIe delivers

- A CUDA kernel addresses **the GPU's own memory and nothing else**. So every frame is copied in (**H2D, host to device**) and every result copied back out (**D2H**). Those two names are what the next five slides are about.



The asymmetry is the whole talk. Host DRAM runs at ~71 GB/s and VRAM at 1 008, but everything between them crawls through **31.5**.

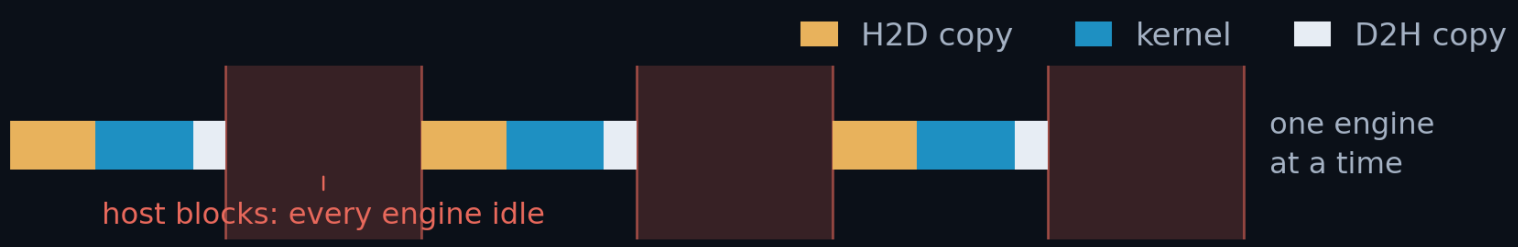
The first port runs at 26 % of the GPU's floor

- One copy in, one kernel, one copy out. The calls are **async**, but the clusters have to be **on the host and copied out** before the next frame starts, so **nothing overlaps**.

ClusterFinderCUDAOpt2.hpp · find_clusters()

```
cudaMemcpyAsync(d_frame,    h_src,      H2D, sc.stream);   // 1. frame in
find_clusters_in_single_frame<<<..., sc.stream>>>(...);    // 2. kernel
cudaMemcpyAsync(h_clusters, d_clusters, D2H, sc.stream);   // 3. clusters out

cudaStreamSynchronize(sc.stream);                          // 4. host stops
for (c : found) out.push_back(c);                          // 5. copy out
```



PCIe is full-duplex: H2D, kernel and D2H run on independent engines and overlap, so the **floor** is **max(H2D, kernel, D2H)**, never the sum. At 3×3 that is **16.2 µs → 61 859 FPS** [s4].

OPT1 · 3×3 · 100 K FRAMES · F64

THROUGHPUT

15 807 FPS

VS 24-THREAD CPU

×2.34

PER FRAME

63.3 µs

H2D · KERNEL · D2H [S4]

16.2 · 15.2 · 7.7 µs

% OF FLOOR · 61 859 FPS

26 %

[s1] opt1's own, uncontended: 13.1 · 14.7 · 5.3 µs

Four streams, but only four frames deep: ×1.56

- A **stream** is an ordered queue of GPU work. Work in **different** streams may overlap, so a copy can run while another stream computes.

- Each stream owns its **own buffers and pedestal**; frames go **round-robin**.

ClusterFinderCUDAOpt2.hpp · per-stream state

```
struct StreamContextOpt2 {
    cudaStream_t   stream;      FRAME_TYPE *h_frame;  // pinned
    FRAME_TYPE    *d_frame;     ClusterType *d_clusters;
    PEDESTAL_TYPE *d_pd_mean, *d_pd_sum, *d_pd_sum2;
};
auto &sc_k = v_sc[k];  // round-robin: frame = round*n_streams + k
```

> **Scaffolding, not yet the payoff.** The streams exist, but the host still synchronises after every round: see opt3.

OPT2 · 3×3 · 4 STREAMS · 4 DEEP

THROUGHPUT

24 726 FPS

VS 24-THREAD CPU

×3.66

PER FRAME

40.4 µs

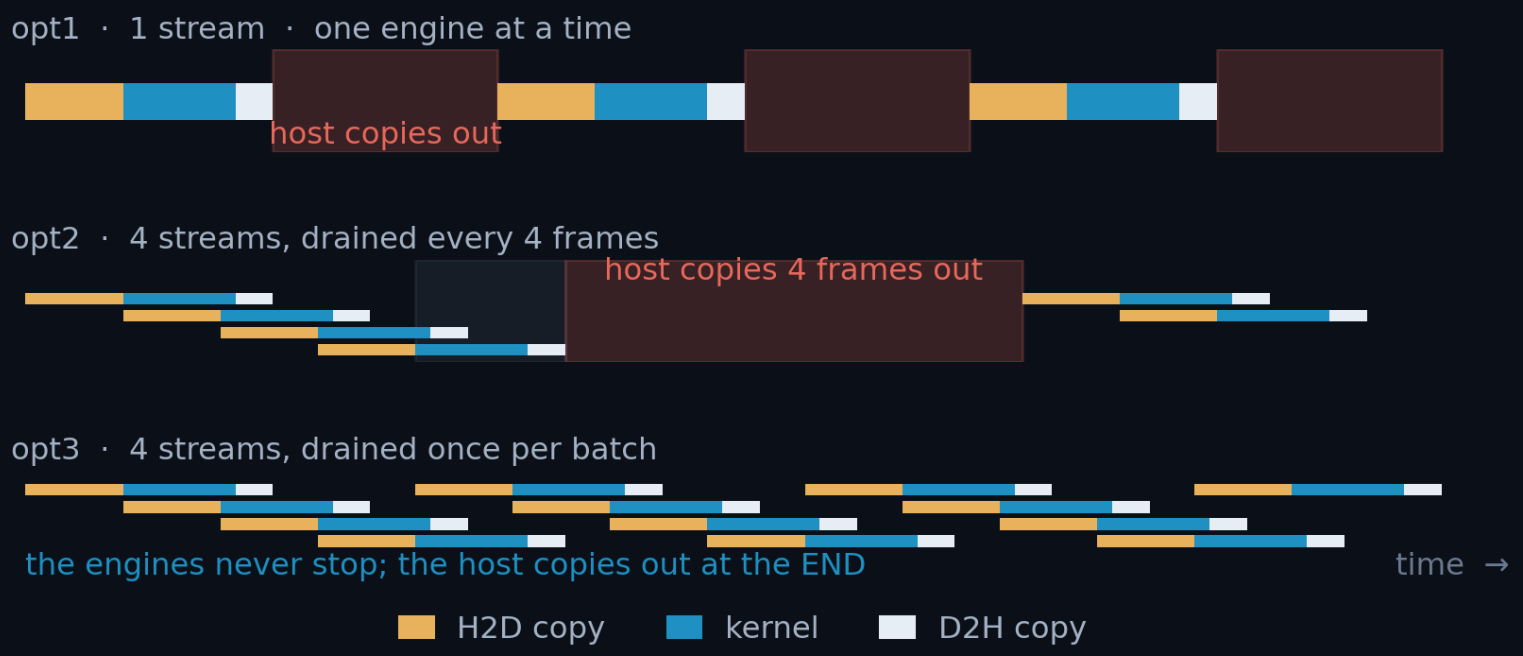
STEP GAIN OVER OPT1

×1.56

% OF FLOOR · 61 859 FPS

40 % (was 26)

One sync per batch, not one per round: ×1.18



THE BARRIER, BEFORE AND AFTER

```
// opt2:  a drain after EVERY round of 4
for (round) {
   for (k : 4) submit(frame);
   for (k : 4) cudaStreamSynchronize(sc[k]);
}
// opt3:  submit the whole batch, drain once
for (frame : batch) {
   cudaMemcpyAsync(..., sc.stream);
   kernel<<<..., sc.stream>>>(...);
   cudaMemcpyAsync(..., sc.stream);
}
for (k : 4) cudaStreamSynchronize(sc[k]);
```

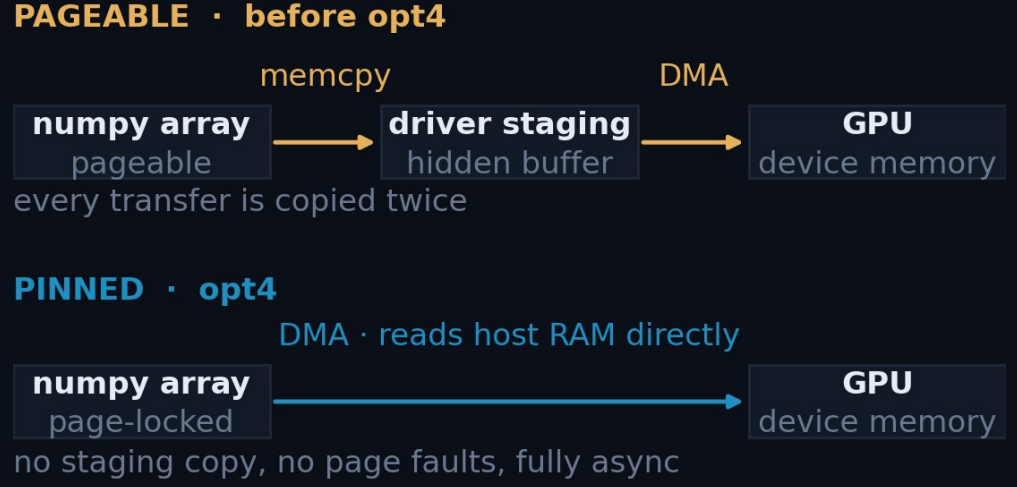
29 188 FPS · ×4.32
34.3 µs/frame · 47 % of floor (was 40)

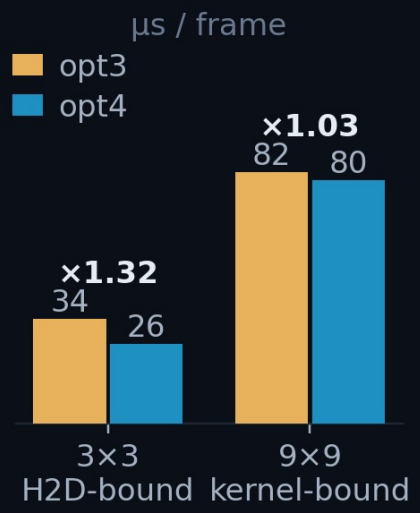
- opt2 drained **all four streams after every round**. The host then copied four frames out, with every engine idle.
- opt3 submits every frame's H2D → kernel → D2H **asynchronously** and synchronises **once per batch**.

Each lane is one stream; H2D and D2H are one FIFO engine each. The red blocks are the host: 48 % of opt1's frame, 46 % of opt2's round. opt3 removes a second, per-frame barrier as well: annex A3.

Pinning the input buys DMA-speed H2D: ×1.32

- Normal host memory is **pageable**, so the driver stages every transfer through a **hidden pinned buffer**: copied twice.
- **Pinning** locks the pages in RAM, so the DMA engine reads host memory **directly**.





ClusterFinderCUDA.hpp

```
// pin the whole dataset once
cudaHostRegister(ptr, bytes,
                 cudaHostRegisterDefault);

// ... run the whole campaign ...

cudaHostUnregister(ptr);
```

38 486 FPS · ×5.69
26.0 µs/frame · 62 % of floor, the largest step in Act I

Measured H2D: one 312.5 KiB frame in **13.2 µs = 24.2 GB/s**, which is 77 % of PCIe 4.0 ×16: true DMA speed. **Pageable staging manages ~15 GB/s** for the same frame, because it is copied twice.

ACT II OF III · GET THE RESULTS BACK

Bottleneck: the result copy, host to host

Both transfers are DMA already. What is slow is what happens AFTER the D2H: copying clusters out of the finder's pinned buffer into heap the caller owns. Host to host. Still 3×3, but 9×9 is where this act pays most.

ARRIVING AT

38 486 FPS

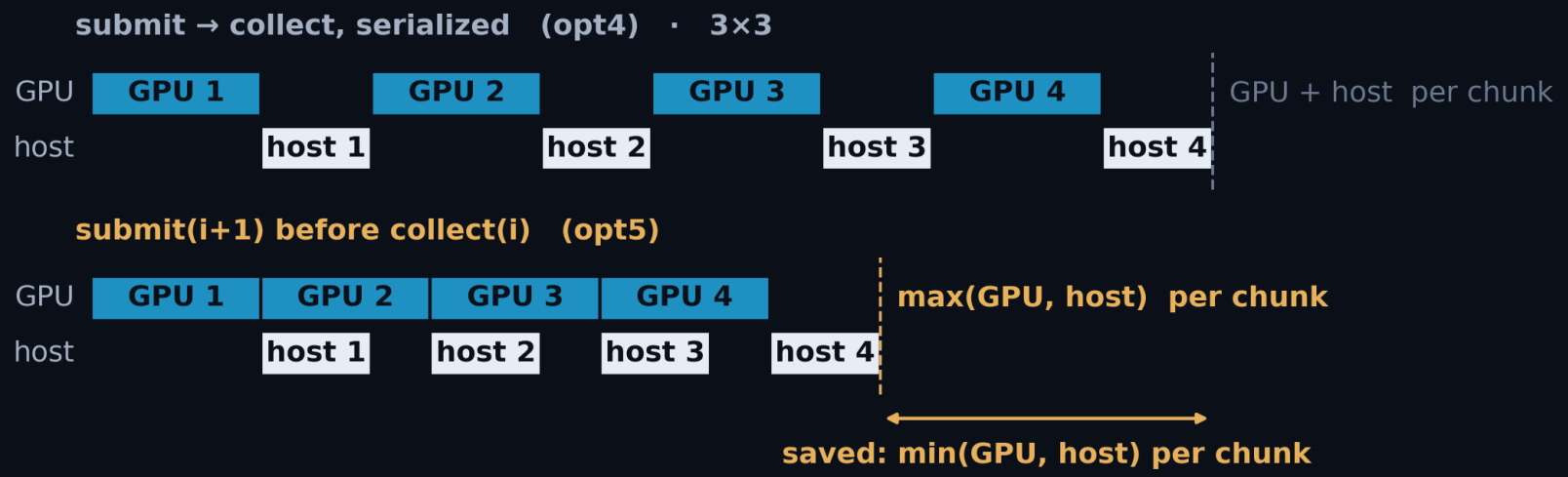
opt4 · 26.0 µs per frame · 62 % of the GPU floor

COMING UP

16	opt5 · host↔GPU overlap
17	9×9 · why overlap runs out
18	opt6 · zero-copy

Overlapping host and GPU hides min(host, GPU): ×1.31

- opt3 overlapped the GPU's own engines, never the **host** with the GPU: the batch synchronised, then built ClusterVectors with the GPU idle.
- opt5 keeps **one batch in flight while materialising the previous one**.



you never write these: find_clusters_batched() wraps them

```
tok = cf.submit_batch(data[a0:b0], first_frame=a0)
for a, b in bounds[1:]:
    nxt = cf.submit_batch(data[a:b], first_frame=a)   // GPU starts i+1
    results.extend(cf.collect(tok))                   // host unpacks i
    tok = nxt
```

OPT5 · F64 · NO CUDA WORK AT ALL

3×3 THROUGHPUT

50 410 FPS

PER FRAME · STEP · OF FLOOR

19.8 µs · ×1.31 · 81 %

THE WHOLE CHANGE

CUDA API CALLS ADDED

none

WHAT MOVED

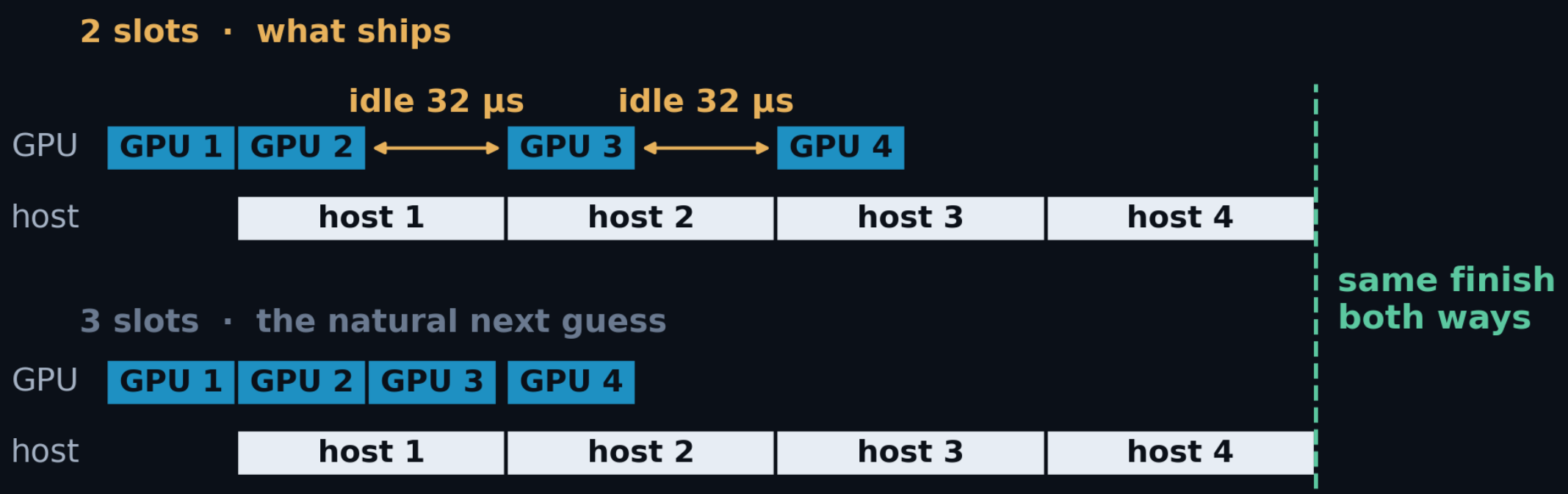
the host loop

9×9 · SAME CHANGE, HALF THE CEILING

66.4 µs · ×1.20 · 45 %

Overlap runs out: the host is the taller bar

- At 3×3 the host copy is **shorter than the GPU floor** and hides under it. At 9×9 it is **roughly twice the floor**, so overlap has less to hide: opt5 is worth **×1.20** here against ×1.31 at 3×3.



> A deeper buffer **relocates the GPU's idle, it does not close it**: the host lane is already back-to-back in both strips, so it alone sets the pace. **The only way down is to make the host term smaller.**

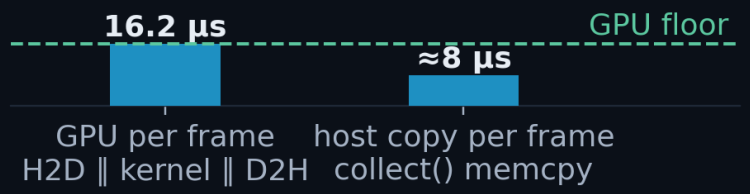
Measured proportions: GPU 30.01 µs/frame, host ~62 µs steady-state [A10].

Read the results in place: ×2.21 at 9×9

- The D2H lands in a **pinned host buffer**; collect() then allocates one ClusterVector per frame and memcpys into it: **467 kB per frame at 9×9**.
- collect_view() returns a **BatchView** onto that buffer instead: it withholds **ownership** past the chunk, not access.

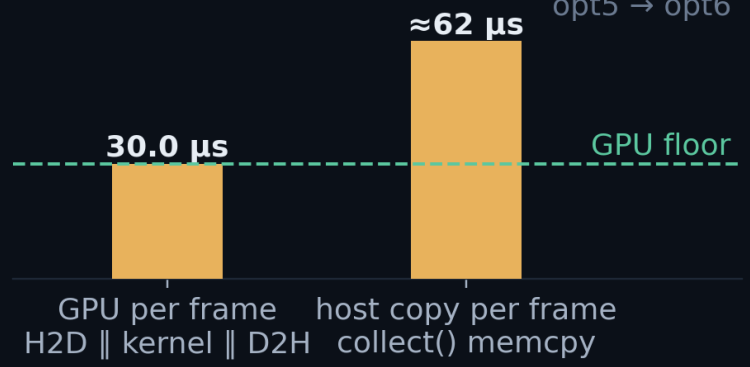
3×3 · host copy 93 kB / frame
the copy hides under the GPU floor → small win

×1.16
opt5 → opt6



9×9 · host copy 467 kB / frame
the copy is larger than the floor → cannot hide

×2.21
opt5 → opt6



> The win is **max(0, host copy − GPU floor)**. At 3×3 the copy already hid under the floor; at 9×9 it is **twice the floor** and cannot hide at any overlap.

OPT6 · 3×3 AND 9×9 · COLLECT_VIEW()

3×3 THROUGHPUT

58 495 FPS

PER FRAME · STEP · OF FLOOR

17.1 µs · ×1.16 · 95 %

9×9 THROUGHPUT

33 323 FPS

PER FRAME · STEP · OF FLOOR

30.0 µs · ×2.21 · 100 %

RESULTS

bit-identical to opt5

ACT III OF III · THE KERNEL

Bottleneck: the kernel itself

The story moves to 9×9, where the kernel is finally the tallest bar, and where most of what it does is update the pedestal, not find photons.

ARRIVING AT

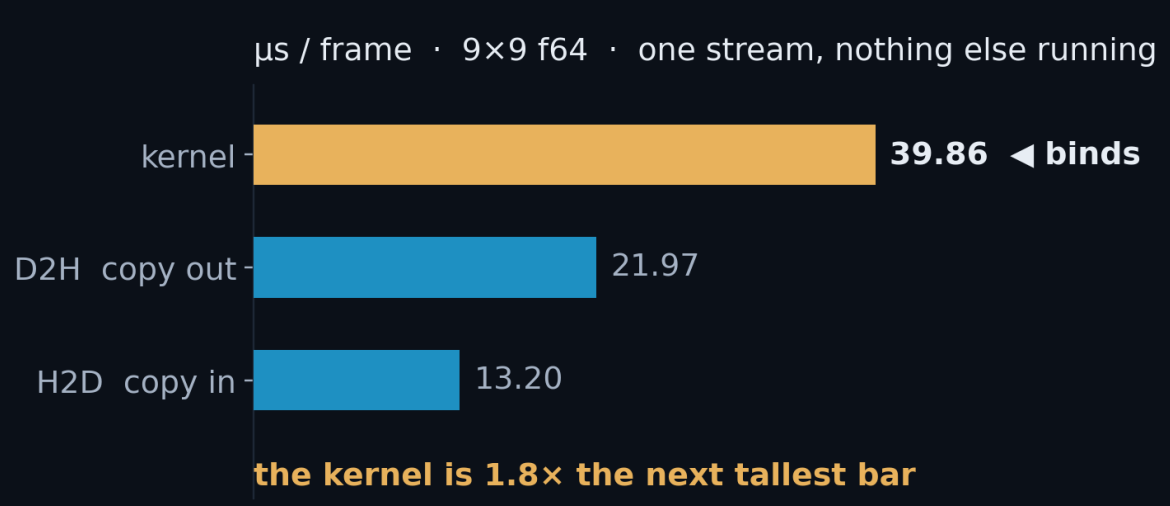
58 495 FPS

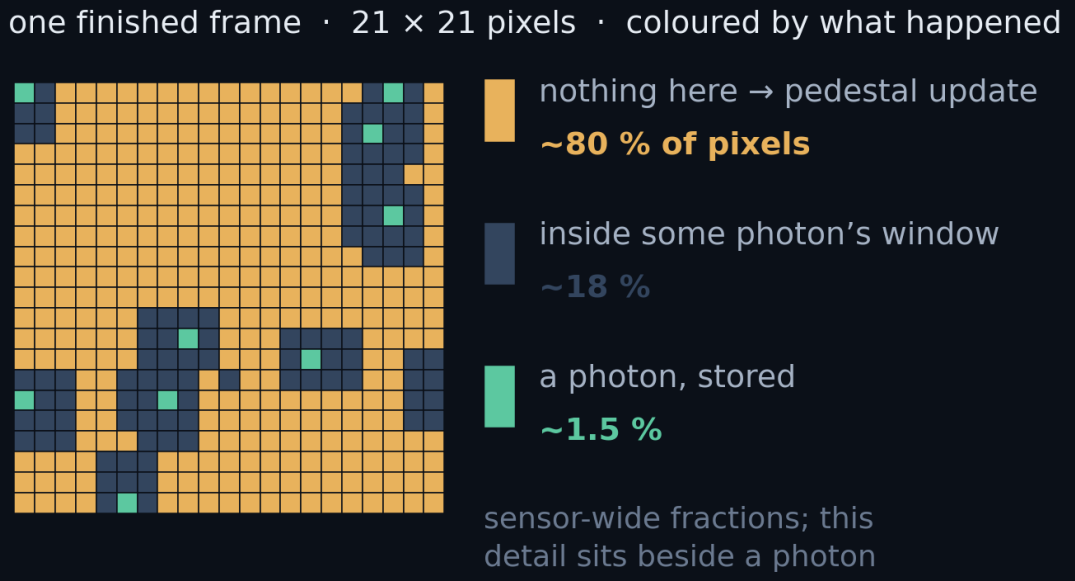
opt6 · 3×3, and 33 323 FPS at 9×9, where this act pays

COMING UP

The kernel is the tallest bar, and 80 % of it is pedestal

- At 9×9, **one stream with nothing else running**, the kernel takes **39.9 µs** against **22.0** for the copy out and **13.2** for the copy in. For the first time in the talk it is the thing to fix.
- **~80 % of pixels see only noise**, and each takes the **pedestal-update** branch: six running values read and written. Most of the kernel is **moving pedestal numbers**, not finding photons.

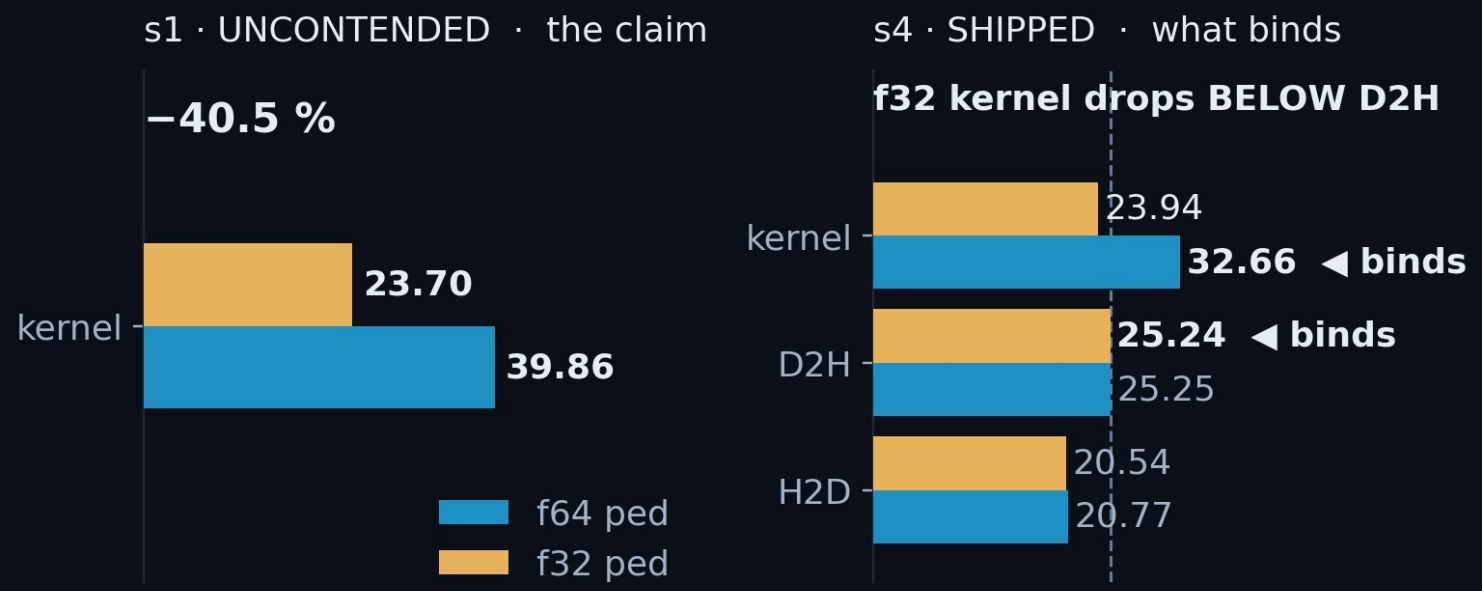




So attack the pedestal, not the photon search. The branch that runs on four pixels in five is the one worth making cheaper.

One typedef: −41 % on the kernel, and the floor moves to D2H

- One typedef halves all four pedestal arrays (**48 bytes per updating pixel in FP64, 24 in FP32**), and the kernel is **bandwidth-bound**.



Left: one stream, nothing else running, so the bar is a true duration and carries the −40.5 % claim. Right: the shipped four-stream pipeline, where the f32 kernel falls under a D2H bar that never moved. Full engine grid: A8.

ONE TYPEDEF

```
// clusterfinder_kernel.cuh
using COMPUTE_TYPE    = float;
using DEVICE_PED_TYPE = float;
//                was: double
```

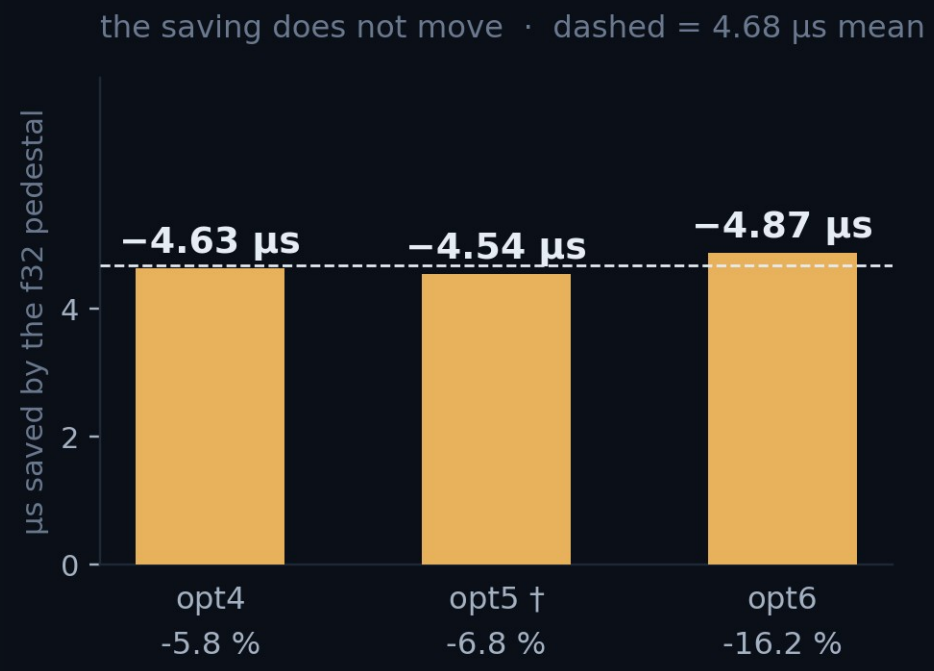
Kernel, 9×9 **uncontended**
39.86 → 23.70 µs (−40.5 %)

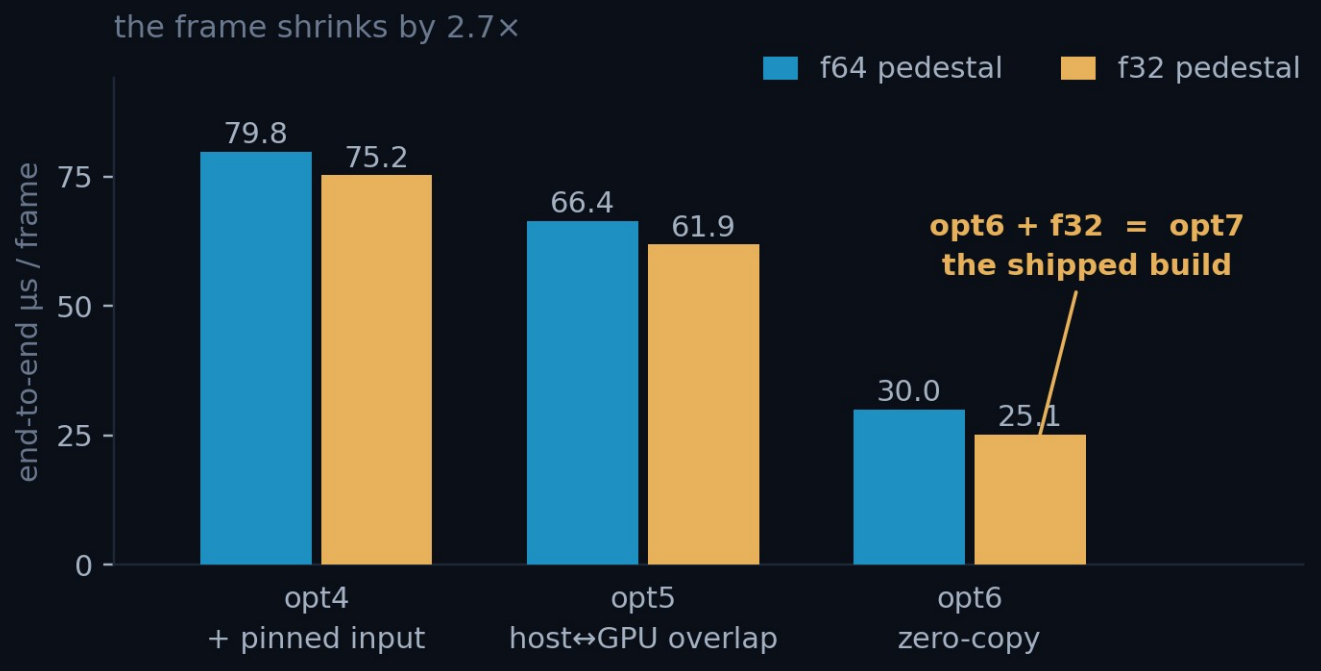
The bottleneck moves. In the shipped pipeline the kernel lands at **23.94 µs**, under a **25.24 µs** D2H. The limit is now the result path, not the maths.

Naive FP32 is **wrong**. See **annex A9**.

Both builds, same git rev, 20 000 frames.

opt7 saves the same 4.7 µs at every step: 6 % of opt4, 16 % of opt6





opt7 ships at 25.1 µs/frame, 100 % of the 9×9 floor. And the act ends by handing that floor away: at **s4** the f64 kernel bound the pipeline at **32.66 µs**; the typedef puts it at **23.94**, under a **25.24 µs** D2H that never moved.

9×9 · 20 000 frames · 4 streams · cap 1 700 · warm · both arms at the same git rev. opt3 excluded and opt5 marked †: their arms sat in different allocator states. Worked through in annex A10.

RESULTS · WHAT CAME OUT OF IT

The full ladder, both cluster sizes

Both cluster sizes end to end, and the audit behind the numbers.

ARRIVING AT

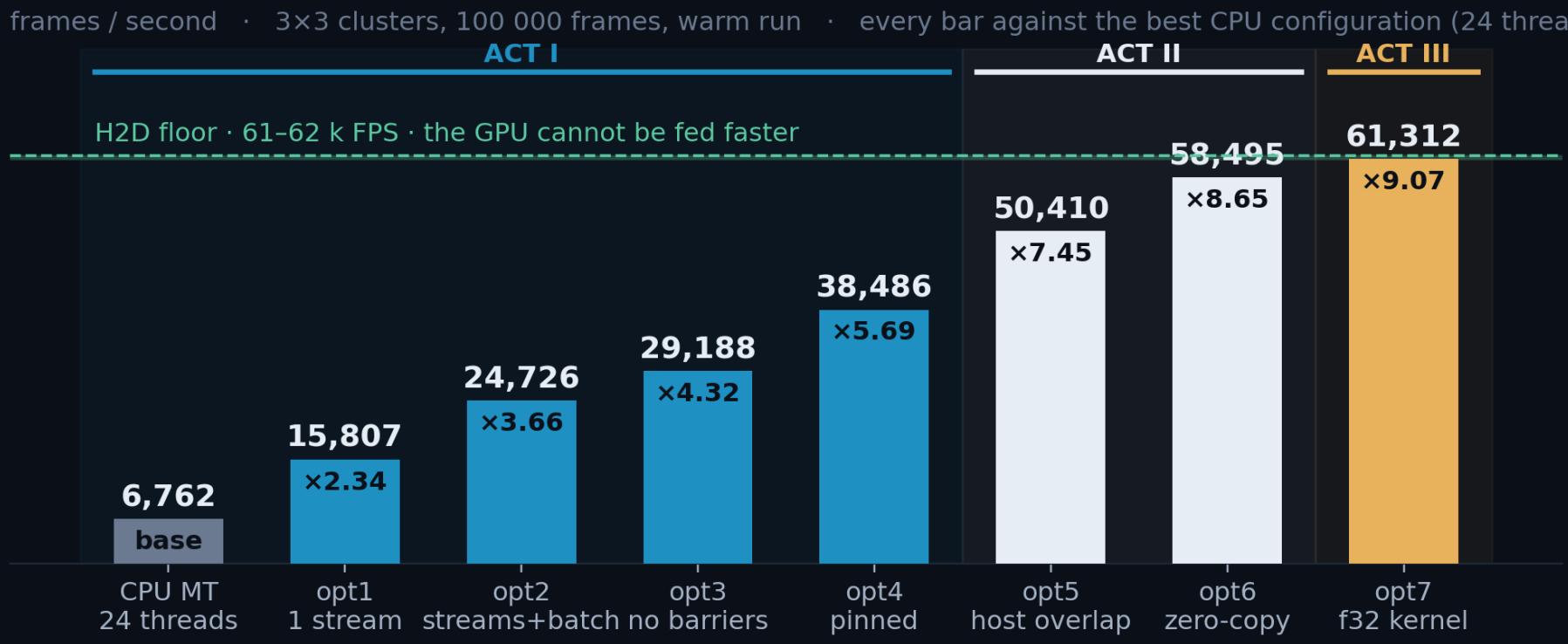
58 495 FPS

opt6 · everything after this is the ladder seen whole

COMING UP

22–23	Results, both cluster sizes
24	Where the time went

×9.1 at 3×3, sitting on the H2D floor

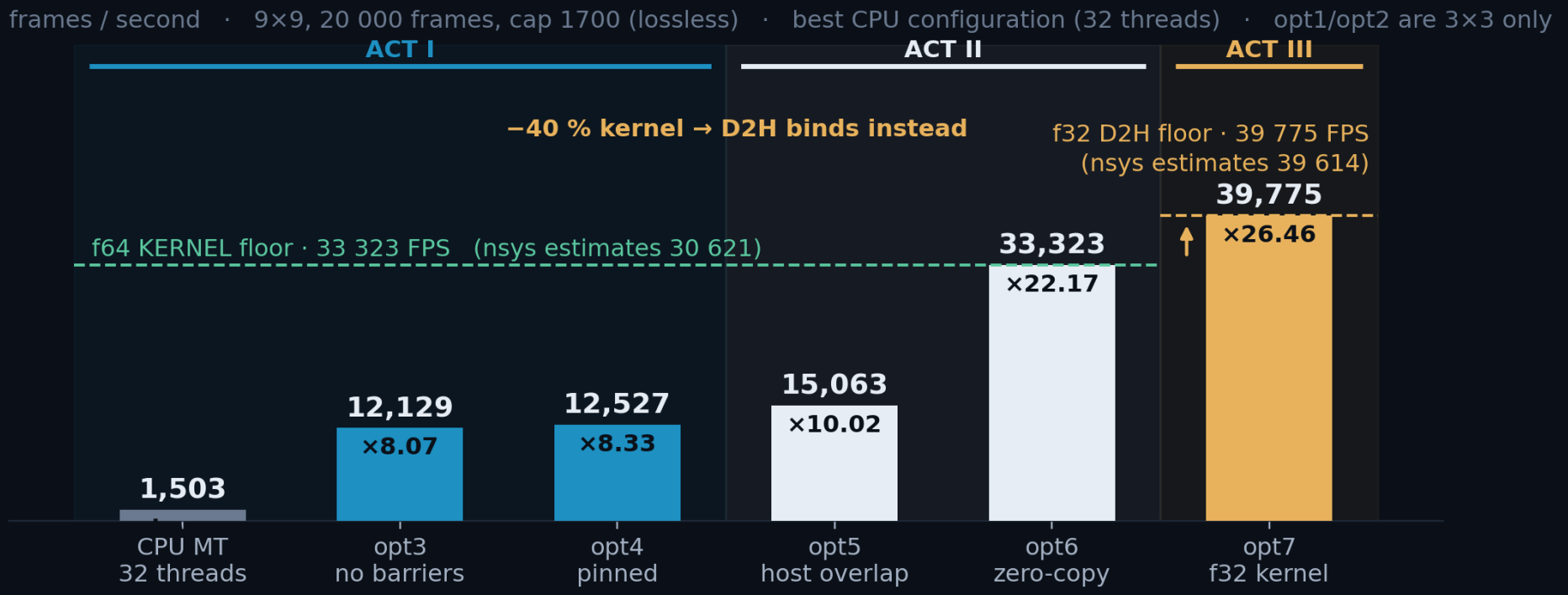


×9.1 over 24 CPU threads, 14.8 s → 1.63 s for 100 000 frames, and **at the H2D floor**.

Every step is **monotonic**, and correctness held constant throughout: **exact on the f64 pedestal**, **6 clusters in 23 M** on the shipped f32.

3×3 · nσ = 5 · 100 000 frames · batch 2 000 · 4 streams · warm = best of reps 1–4 · each step in its own process.

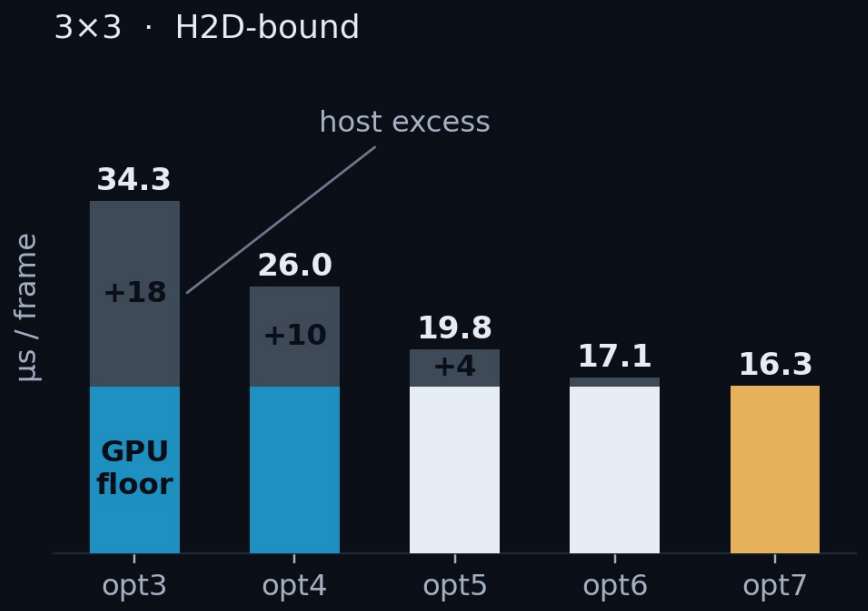
×26.5 at 9×9, and opt7 hands the floor to D2H

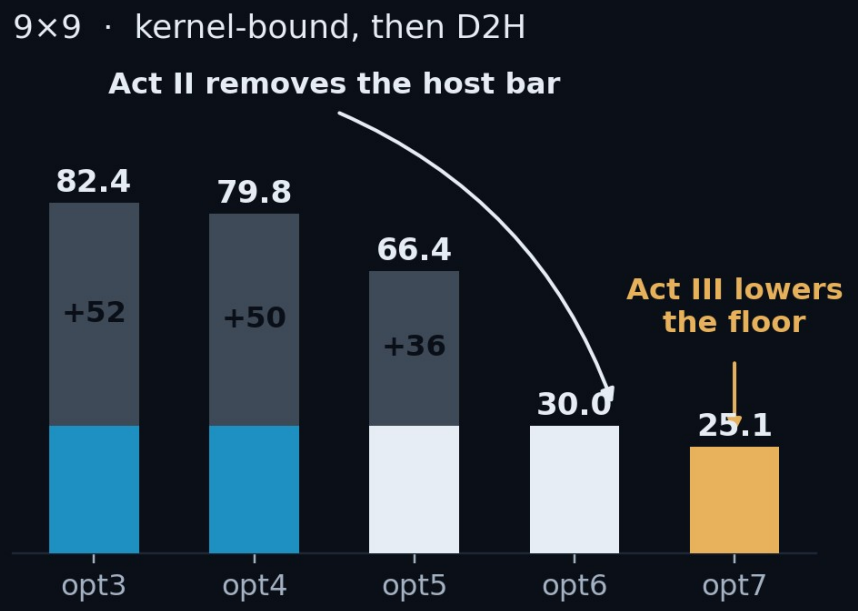


×26.5 over 32 CPU threads; the kernel is the tallest bar for the whole f64 arm, and opt7's −40 % drops it **below D2H**.

9×9 · cap 1 700 · 20 000 frames · CPU baseline at 32 threads. opt1 and opt2 are 3×3-only. Why the two arms differ is in the notes.

The host bar dies first, then the floor itself drops





Acts I and II never touch the arithmetic. The GPU floor is a flat 16.2 µs at 3×3 / 30.0 µs at 9×9; what collapses is everything stacked on it.

Act III is the only step that lowers the floor itself, and it could not have been seen until the stack above it was gone.

Coloured = the GPU floor for that act's build. Grey = everything the host adds on top: +50 µs at opt3, nothing at opt6.

Does it find the same clusters?

Whether the CUDA finder returns the same clusters as the CPU.

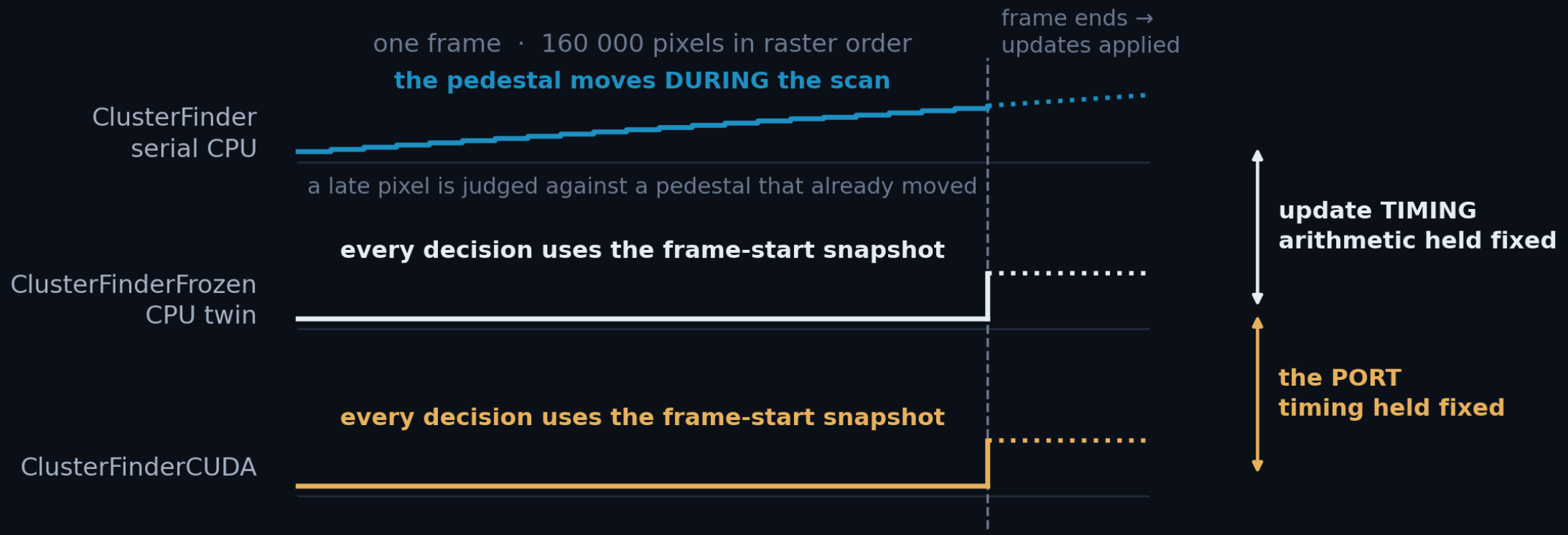
ESTABLISHED

×9.1 and ×26.5

on the hardware floor at both cluster sizes, if the physics holds

COMING UP

CPU and CUDA update the pedestal at different moments



The CPU finder updates the pedestal **as the raster reaches each pixel**. 160 000 CUDA threads read it at once, so the update is **applied at the frame boundary**.

A straight CPU↔CUDA comparison therefore moves **two** things at once. **ClusterFinderFrozen** moves only the update: same arithmetic, same gates, same scan.

Frozen is a diagnostic twin, not a product. Every finder trains on the same 1 000 pedestal frames and runs the same 10 000 data frames.

CUDA and its CPU twin agree exactly: 0 in 23 million

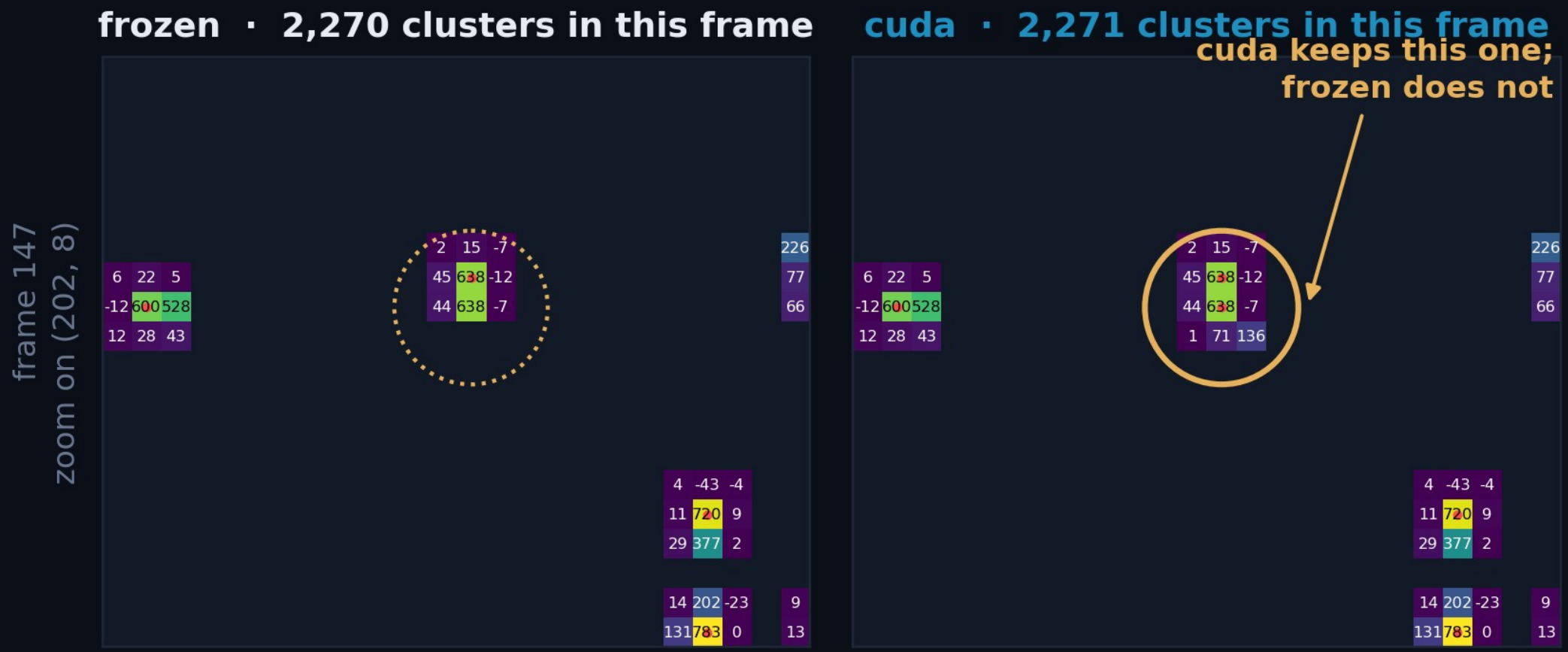
- **ClusterFinderFrozen** makes byte-for-byte the same decisions as ClusterFinder and differs in exactly one thing: **when** the pedestal is updated. Frozen per frame, pushed at frame end. That is the CUDA model, so comparing against it isolates everything else the port changes.

COMPARISON	THE ONE THING THAT DIFFERS	A-ONLY / B-ONLY	% OF CLUSTERS
frozen CPU vs CUDA [f64 ped]	**nothing**	**0 / 0**	**0 %**
frozen CPU vs CUDA [f32 ped]	the float32 pedestal EMA drifting: see next slide	0 / 6	0.000026 %

> **The ringed row is the port on its own.** With the update timing held identical, the two finders agree on all **23 244 605** clusters. Not small: zero. The six at f32 are the next slide.

3×3 · 10 000 frames · 23.2 M clusters · exact centre-set difference at tol = 0. [f64 ped] and [f32 ped] differ only in DEVICE_PED_TYPE. What is left once the timing is allowed to differ is slide 29.

The whole disagreement is one duplicate centre



Only each finder's **own** 3×3 footprints are drawn, so the panels differ **exactly** where the finders do.

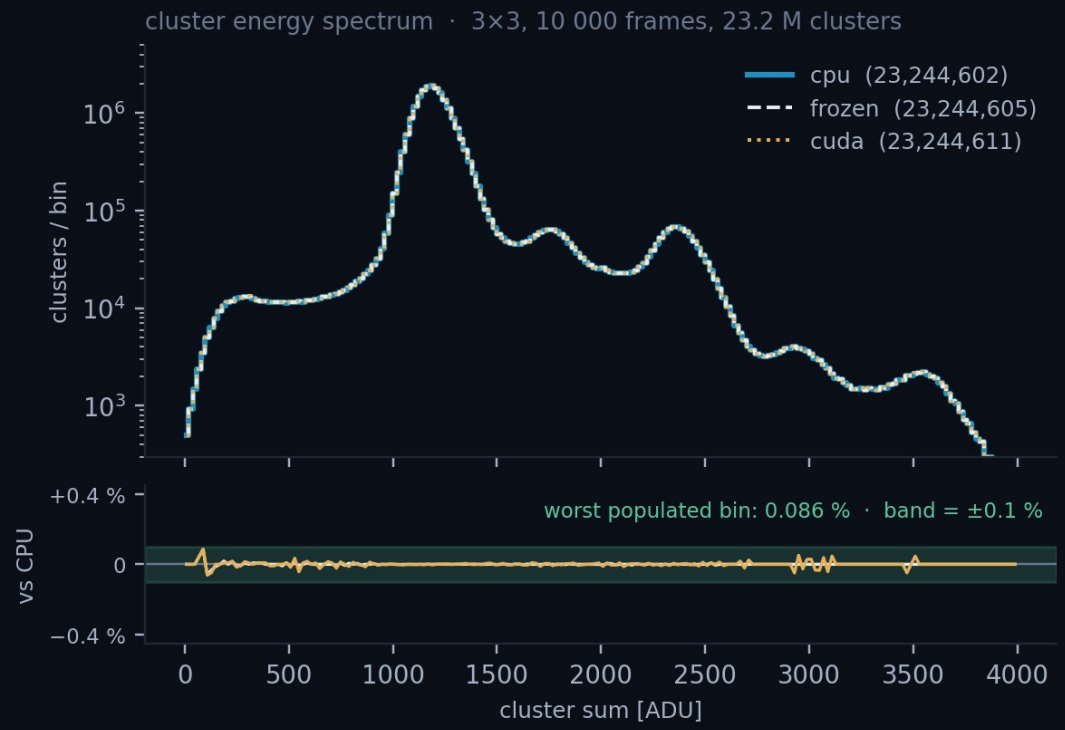
cuda's patch is **one row taller**: a second centre directly below the one both found. **The charge is already counted**: a duplicate, not a new photon.

float32 cannot tell these two pixels apart

```
frame 147    centre (x=202, y=8)      3×3 window
raw window (ADU)     pedestal-subtracted, 1 decimal
[[4646 5282 4703]    [[ 45.3 638.4 -12.1]   frozen and cuda
 [4857 5318 4950]     [ 43.7 638.4  -7.5]   print the SAME
 [4763 4640 4858]]    [  1.1  70.7  136.4]]   window

the two contenders, at full precision:
                        rival (dy=-1)     centre
frozen   [f64 ped]     638.383019956   638.382773664
cuda     [f32 ped]     638.382812500   638.382812500

gate:  accept if  centre >= max(window)
   frozen   638.382773664 >= 638.383019956  ->  reject
   cuda     638.382812500 >= 638.382812500  ->  accept
```



Under the f64 pedestal the two pixels are **0.000246 ADU** apart; one float32 **ULP*** here is **0.000488**. **Half a ULP**: in float32, the same number.

At f64 the residual is zero.

* ULP = unit in the last place, the gap between one float32 and the next. Here it is 2^{-11} = 0.000488 ADU, set by the ~5 000 ADU operands of the subtraction, not by the 638 ADU answer, whose own ULP is 8× finer.

All 19 clusters come from when the pedestal is updated

- The serial finder **updates the pedestal as it scans**, so a pixel can be tested against a pedestal its own neighbours already moved. Frozen and CUDA both **hold the pedestal still for the whole frame**.
- That is the only thing separating these two rows from the exact pair. And the rows carry **the same 8 and 11 clusters**, so **nothing CUDA does contributes anything at all**.

COMPARISON	THE ONE THING THAT DIFFERS	A-ONLY / B-ONLY	% OF CLUSTERS
serial CPU vs frozen CPU	update timing alone: **a CPU-only effect**	**8 / 11**	0.000082 %
serial CPU vs CUDA [f64 ped]	the same thing, **and nothing else**	**8 / 11**	0.000082 %

> **19 in 23 244 605, and the port is in none of them.** frozen and CUDA agree exactly at f64, so the one variable left between serial and CUDA is **when the pedestal moves**. Which test it moves is the next two slides.

3×3 · 10 000 frames · 23.2 M clusters · exact centre-set difference at tol = 0. A-only means found by the left finder and not by the right.

There is a third test

- Slide 4 showed the decision with **two** tests, which is the right simplification for the arc. The finder has **three**, and the missing one is what the next slide is about.

ClusterFinder.hpp:146-186 · all three branches

```
v = frame[i] - ped_mean[i];   rms = ped_rms[i]
if (v < -nSigma*rms)           -> skip, no update
m     = max over the 3x3 window
total = sum over the 3x3 window
if (m > nSigma*rms)              // TEST 1
    if (v == m) -> emit cluster  //   local-max gate
    else        -> shadow, no update
else if (total > c3*nSigma*rms)   // TEST 3
    if (v == m) -> emit cluster
else            -> update pedestal
```

WHAT EACH TEST ASKS

TEST 1 · NSIGMA · RMS

is any ONE pixel bright?

TEST 3 · C3 · NSIGMA · RMS

is the WINDOW bright?

LOCAL-MAX GATE

is this pixel the peak?

Test 1 reads the window's MAX; Test 3 reads its SUM. That one difference is the next slide.

c3 = √(3×3) = 3 falls out of variance addition. Variances add, standard deviations do not: nine pixels of noise σ give **Var(Σ v) = 9σ²**, so the sum's noise is **3σ**. **c3·nSigma·rms** is therefore the **same 5σ criterion as Test 1**, asked of the window instead of the pixel.

Roughly 80 % of pixels reach the last branch and push the pedestal; ~1.5 % are peaks and ~18 % sit in a peak's shadow.

Test3 makes the clusters; Test1 only moves the pedestal

- **push_fast** touches only the pixel's **own** accumulators and never reads the stencil, so the update is **order-independent**: same set of updated pixels, bit-identical pedestal. Only a differing **decision** can diverge.

the finished frame · 21 × 21 around (125, 245)

- stored: this pixel IS the window max, and it clears 5σ
- shadow: the window max clears 5σ, but this pixel is not it
- pedestal sample: nothing in the window clears 5σ
- the disputed pixel: serial sampled here, frozen stored

the moment (125, 245) was tested · only the 4 already-scanned neighbours differ

31.669 31.637	-14.420 -14.405	33.763 33.730
41.680 41.638	56.473	30.007
15.214	40.217	21.979

frozen above serial · amber ring = pushed pedestal

serial Σ = 256.489 < 256.511 → pedestal sample

frozen Σ = 256.581 > 256.511 → CLUSTER

max = 56.473 in both → Test1 never moved

> Measured two ways. **Instrumented**: of the 19 disagreeing clusters, the **11 that only frozen finds are all Test 3**; the 8 only serial finds are all the local-max gate. **Ablated**: compile Test 3 out and the 11 go to **zero**.

3×3 · 10 000 frames · 23 244 605 clusters · per-pixel branch codes diffed frame by frame, then re-run with Test 3 compiled out.

The fast path in eight lines

THE RECOMMENDED PATTERN

```
from aare import File, ClusterFinderCUDA
cf = ClusterFinderCUDA(image_size=(400, 400), cluster_size=(3, 3),
                       n_sigma=5, n_streams=4,
                       max_clusters_per_frame=3000)
for _ in range(1000):                    # 1. train the pedestal
    cf.push_pedestal_frame(pd.read_frame())
data = f.read_n(100_000)                 # 2. one contiguous array
cf.register_input_buffer(data)           # 3. pin it once
for s in range(0, len(data), 2000):      # 4. batch through it
    clusters = cf.find_clusters_batched(data[s:s+2000], first_frame=s)
cf.unregister_input_buffer()             # 5. release the pages
```

- **3 · register_input_buffer()**: opt4 in one call. Pin **once**, outside the loop.
- **4 · find_clusters_batched()**: one ClusterVector per frame, with opt5's overlap built in.

> **Zero-copy: ×1.16 at 3×3, ×2.21 at 9×9.** The views expose every cluster; they only withhold **ownership** past the chunk.

ZERO-COPY · SAME PATTERN, STEP 4 SWAPPED

```
for v in cf.find_cluster_views_batched_iter(data, 2000):  # 4, zero-copy
    hist.fill(v.sums())        # consume inside the loop:
                               # the view dies with its chunk
```

> Reduce as you go and you never allocate. Hold a view past its chunk and you stall the pipeline.

Five knobs, and the one that silently truncates

> The single most common mistake: leaving **max_clusters_per_frame** too low. It does not error; it truncates, and every frame quietly returns the same count.

PARAMETER	WHAT IT DOES	GUIDANCE
`n_streams`	Upper bound on frames in flight.	4 at both cluster sizes. 8 buys no kernel concurrency at 9×9.
`max_clusters_per_frame`	Fixed size of the per-frame D2H transfer.	Must exceed the real maximum or clusters are silently dropped. At 9×9 the maximum is 1 633, and a cap of 1 700 already makes D2H the bottleneck.
`batch size`	Frames per find_clusters_batched call.	2 000 amortises launch overhead without a large pinned footprint.
`cluster_size`	Compile-time stencil geometry.	3×3 and 9×9 are registered. 9×9 moves the bottleneck off H2D.
`register_input_buffer`	Page-locks the host array for DMA.	Always, if the data is already in RAM.

WHERE THIS LEAVES US

The bottleneck has walked from the host, to the GPU, to PCIe

Every number in this deck is measured on one machine, one dataset, one git revision, and the annexes carry the reconciliation for each of them.

DONE

×9.1 at 3×3, ×26.5 at 9×9

16.3 and 25.1 µs/frame end to end, both on their hardware floor. At 3×3 that is 47× MÖNCH03's standard frame rate.

DONE

FP32 pedestal, safely

−40.5 % kernel time, and correct: the variance accumulates on a frozen per-pixel offset. 6 duplicate clusters in 23 million.

NEXT

3×3: transfer granularity

The pipeline sustains 16.31 µs against a 13.15 µs uncontended H2D: 2 000 separate 320 kB descriptors, plus H2D↔D2H contention.

NEXT

9×9: the result path, not the kernel

At a lossless cap D2H already binds: 25.24 µs against a 23.94 µs kernel. More kernel work buys nothing.

Annexes

EVERYTHING SO FAR

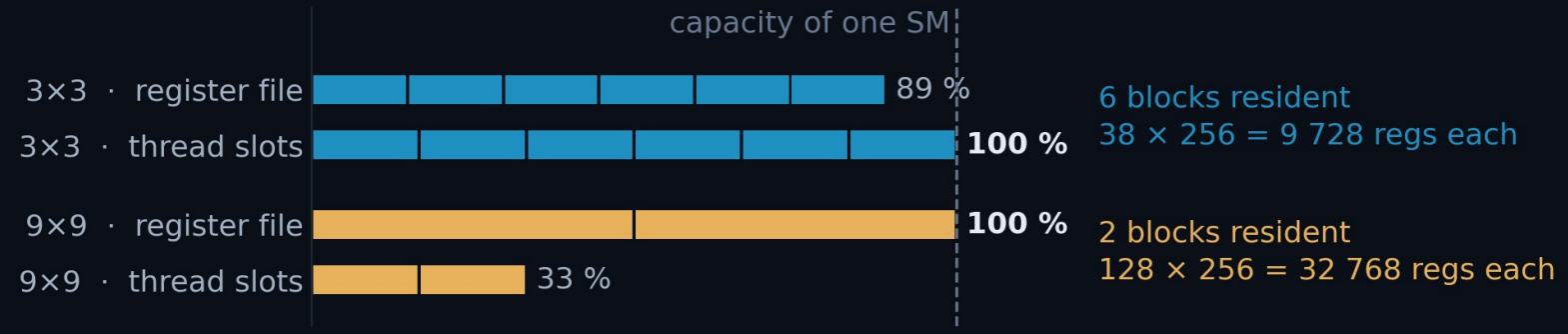
33 slides

the arc is finished; what follows answers questions

COMING UP

Register pressure: 38 per thread at 3×3, 128 at 9×9

- An SM has **65 536 registers** and **1 536 thread slots**; whichever runs out first decides how many blocks fit at once.
- Every thread keeps a private **clusterData[CSX × CSY]**, so register demand grows with the **square** of the cluster size. Neither build spills.



> **9×9:** the register file is exactly full at two blocks (2 × 128 × 256 = 65 536). **3×3:** the slots run out first, and the registers still have room.

MEASURED, NOT ESTIMATED · READ FROM THE BUILT .SO

```
cuobjdump -res-usage build/aare/_aare_cuda*.so | c++filt
  3x3: REG:38  STACK:0 LOCAL:0     # STACK/LOCAL 0 = no spills
  9x9: REG:128 STACK:0 LOCAL:0
```

PER SM · SM_89 · F32 BUILD

3×3 · BLOCKS RESIDENT

6

9×9 · BLOCKS RESIDENT

2

SPILLS, EITHER CASE

0 bytes

3×3 ON THE F64 BUILD

47 regs → 5 blocks

CUDA Graphs, a sound idea that the next act overtook

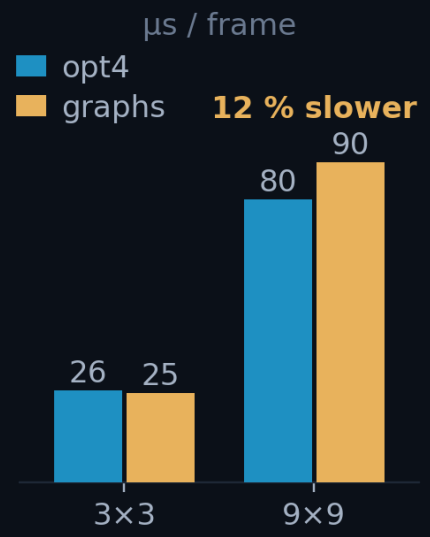
- Every cudaMemcpyAsync / kernel launch costs the **CPU** a few microseconds of driver work, per frame and per operation. After opt4 that looked like the budget.
- A **CUDA Graph** builds the whole dependency DAG once, so replay should be **one** cudaGraphLaunch. **Ours needs three per frame**, because every frame has new source and destination addresses.

WITHOUT GRAPHS · one driver call per operation

CPU cost
6 calls

WITH GRAPHS · the ideal: replay with one launch

CPU cost
1 call



ClusterFinderCUDA_graph.hpp

```
// setup: nodes added by hand, not captured
cudaGraphCreate(&sc.graph, 0);
cudaGraphAddMemset/Memcpy/KernelNode(...);
cudaGraphInstantiate(&sc.graphExec, ...);

// per FRAME: 3 driver calls, not 1
cudaGraphExecMemcpyNodeSetParams(...);  x2
cudaGraphLaunch(sc.graphExec, sc.stream);
```

REJECTED
3×3: 39 752 FPS, inside noise of opt4.
9×9: 11 072 FPS, 12 % slower.

The 3×3 edge was never established: the instrumentation tax (2.8 µs) is larger than the gap (0.8 µs). Detail in the notes.

What a CUDA Graph actually saves, in microseconds

- The stream path issues **four runtime calls per frame** (memset, H2D, launch, D2H) and a graph replaces all four with **one** cudaGraphLaunch. This is the **ceiling**: our own graph needs **three** (see part 1).

CALL	PER FRAME	HOST COST	µS/FRAME
cudaMemcpyAsync	2	1.98 µs	3.97
cudaLaunchKernel	1	2.13 µs	2.13
cudaMemsetAsync	1	1.57 µs	1.57
submission total	**4**		**7.67**

THE ARITHMETIC

```
7.67 us/frame x 3/4  = 5.75 us/frame   eliminated, AS MEASURED (under nsys)
5.75 / 4 (see A12)   ~ 1.4 us/frame    eliminated, unprofiled estimate
```

~1.4 µs against a 16.17 µs floor = 8.7 %, real while the host is the critical path, and **worth nothing after opt5**, which hides host work under the GPU entirely.

ROUTE A · MEASURED VERDICT

3×3 VS OPT4

×1.03

9×9 VS OPT4

×0.88

VS OPT5 AT 9×9

36 % behind

Read from CUPTI_..._RUNTIME, the host table, so this is the one order-of-magnitude number in the deck, ±50 %. It is enough to show graphs cannot pay against a 16 µs floor, and not enough to quote to two figures.

One D2H per frame, not two

- opt2 asked the device **how many clusters**, blocked until the answer came back, then asked for **that many**. The size of the second copy was a function of data that had not arrived yet.
- opt3 gives every frame a **fixed envelope** (count, then room for **cap** clusters), so the copy's size is known at construction and can be queued with the kernel. The count is still read, but **afterwards**, on the host.

1 kernel › 2 copy 4 B › 3 BLOCK › 4 read count › 5 copy N B › 6 BLOCK

opt2 · two transfers and two stalls per frame: the host must learn the count before it knows how much to fetch.

1 kernel › 2 copy the whole envelope › 3 → next frame, host not involved

opt3 · one transfer, no stall. Nothing in the loop waits on a value.

You can only stream a transfer whose size is known in advance. The price is bytes: the envelope is sized by the **cap**, not by the clusters actually found. **3×3: 120 kB shipped for 93 kB of clusters. 9×9: 558 for 467.**

The overlap, in six lines, and why you never write them

THE WHOLE OF OPT5

```
tok = cf.submit_batch(data[a0:b0], first_frame=a0)
for a, b in bounds[1:]:
    nxt = cf.submit_batch(data[a:b], first_frame=a)   # GPU starts N+1 …
    results.extend(cf.collect(tok))                   # … host unpacks N
    tok = nxt
results.extend(cf.collect(tok))                       # drain the last one
```

- **You do not write this.** It is inside find_clusters_batched(), so opt5 arrived as a **speedup, not an API change**.
- submit_batch() and collect() stay public for streaming from a detector, or interleaving other work between chunks.

> Two chunks in flight is enough. A third adds pinned memory and no overlap: the host is already busy for the whole time the GPU is.

The saving is min(GPU, host) per chunk, so opt5 pays most when the two terms are comparable, ×1.31 at 3×3, and least when one dominates, ×1.20 at 9×9, where the host term is roughly twice the GPU term. That gap is the diagnosis that motivates opt6 (report §8.2).

CHUNK SIZING · THE TWO CONSTRAINTS

MULTIPLE OF

n_streams

A multiple of n_streams, because the device pedestal is per-stream: an uneven chunk leaves the four pedestals at different ages and the finder stops being reproducible.

CAPPED AT

MAX_SLOT_BYTES

Capped so the two pinned slots stay bounded: 544.5 kB per frame at 9×9, cap 1 700. chunk_size_for(n) applies both rules.

The copy is allocation-bound, not bandwidth-bound

Copying faster does not help when the cost is the OS populating pages. The only winning move is not to allocate, which is exactly what opt6 does. materialize_slot() is deliberately single-threaded and carries a comment saying so, to stop the experiment being repeated.

B′ One allocation per chunk

`collect_packed()`

Removes the per-frame malloc but keeps the copy, and the copy is ~80% of the cost. Worse, the replacement allocation is 1.17 GB, far above glibc's mmap threshold, so it is mmap'd and munmap'd every chunk: 606 566 faults, ~21 µs/frame.

69.3 µs · deleted from the API

B″ Parallel materialisation

`8-thread copy pool`

Each worker gets its own glibc arena, which destroys the cross-run heap reuse that makes the single-threaded path cheap. Faults went 9 700 → 2 270 000; MALLOC_ARENA_MAX=1 collapsed them back to 138 k, which is the proof.

+6% at best, −33% when results are freed promptly

Why not pin one buffer and keep every cluster?

- **The proposal.** Give the finder a big **pinned** buffer; every frame's D2H lands in it at its own offset. No host-to-host copy, and the results outlive the finder. **It works, and it is faster than opt5.**
- **What binds is not time, it is RAM**, and the input has to stay pinned **at the same time**, since every frame's H2D still reads from it.

9×9 · PINNED AT ONCE	PER FRAME	N = 20 000	N = 100 000
input frames · opt4 already does this	320 000 B	6.4 GB	32.0 GB
output envelope · what this adds	557 568 B	11.2 GB	55.8 GB
both, for the whole run	877 568 B	**17.6 GB**	**87.8 GB**

98 GB free, no swap. At the measured size (N = 20 000) it fits easily. It is **linear in N**: 877 568 B per frame caps a 9×9 run at **≈112 000 frames** with nothing left over.

Faster than opt5, and it costs 20 % more memory than opt5's own results. The envelope stores the **cap** (1 700), not the **1 422** clusters actually found, and all of it is page-locked.

9×9 · PER FRAME

OPT5 · MEASURED

66.39 µs

OPT6 · NO HOST COPY EITHER

30.01 µs · 100 % of floor

RESULTS KEPT, N = 100 000

OPT5 · PAGEABLE, COMPACTED

46.6 GB

THIS · PINNED, PADDED TO CAP

55.8 GB

Registration is in no timed number here: at 3×3 the whole cold–warm gap is 0.097 s, and covering the 32 GB input inside it would need 320 GB/s. Measured DRAM is 71 GB/s.

Four kernels overlap, so the kernel floor is busy time, not run time

- **Why it needs saying.** In Act III the **kernel** is the tallest bar, and it is the one engine that runs **four at a time**. A copy engine's busy time is simply how long it runs; the kernel's is not, so what its floor means stops being obvious.

s1
One stream, nothing else running
How long an operation actually takes.

s4
The shipped pipeline, four streams
How busy each engine is: a union, not a sum.

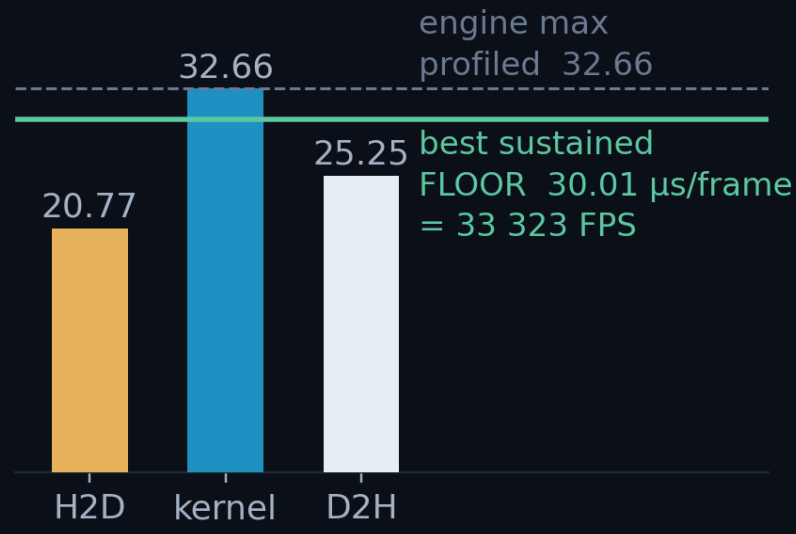
floor
Set by the busiest engine
1 / the busiest engine, in µs/frame or FPS.

s4 · the shipped pipeline, four streams · schematic, 9×9 shape

one copy engine per direction, so the H2D bars queue

UNION: what s4 reports. The kernels overlap, so it is shorter than their sum.

busy µs / frame · 9×9 f64 · measured



Uncontended, or as the pipeline runs it

- Every engine time in this deck is tagged **[build · s1|s4]**. **s1** is one stream with nothing else running: what an engine does **on its own**, which is the right number for a capability claim and for the headroom that remains. **s4** is the shipped four-stream pipeline: each engine's **busy time per frame**, the union of its intervals, the only number that can set a floor.

3×3 · µS/FRAME	S1 F64	S1 F32	S4 F64	S4 F32
H2D	13.14	13.15	16.17	16.63
kernel	14.72	4.32	15.17	5.53
D2H	5.31	5.27	7.69	7.57
engine max [s4]	—	—	16.17	16.63
FLOOR = lower of max, sustained	—	—	**16.17**	**16.31**

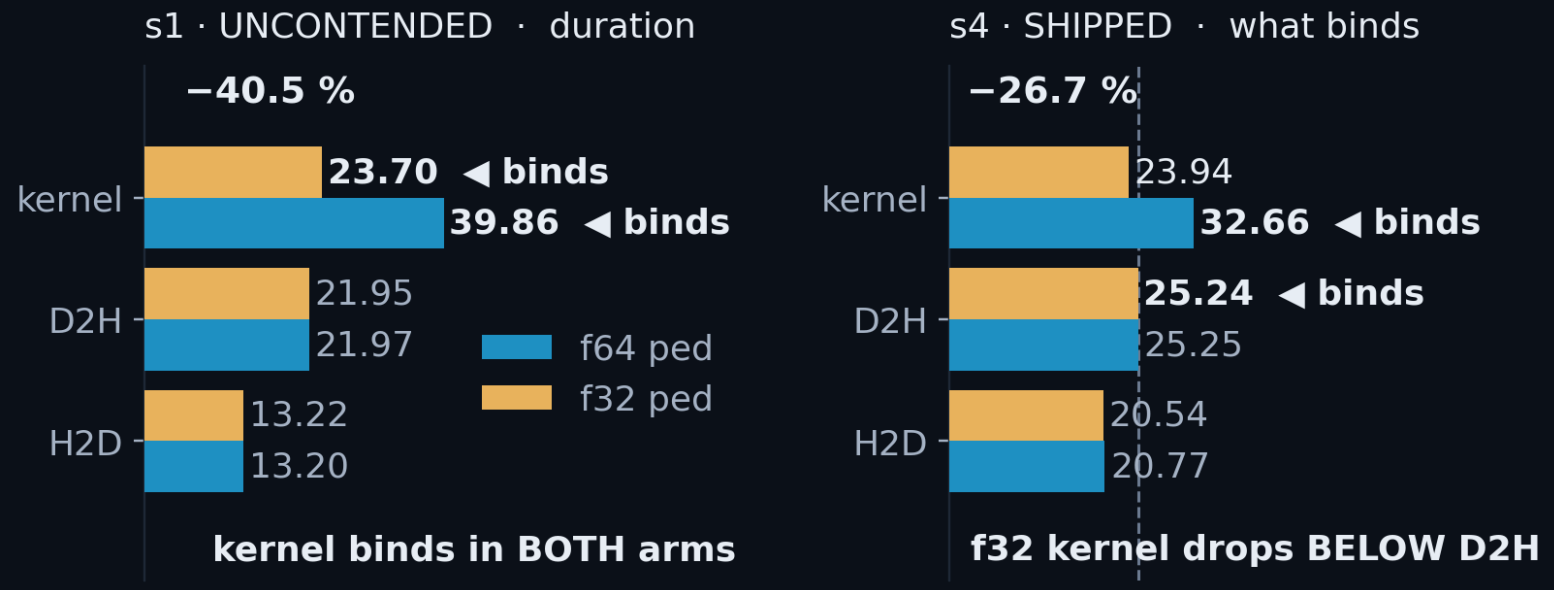
9×9 · CAP 1700	S1 F64	S1 F32	S4 F64	S4 F32
H2D	13.20	13.22	20.77	20.54
kernel	39.86	23.70	32.66	23.94
D2H	21.97	21.95	25.25	25.24
engine max [s4]	—	—	32.66	25.24
FLOOR = lower of max, sustained	—	—	**30.01**	**25.14**

> **Two traps.** (1) s4 is engine *occupancy*, not duration: the 9×9 kernel row **falls** under load. (2) the **floor is not the engine max**: the max is profiled and runs 2–8 % high.

The D2H shift is an s4 phenomenon: at s1 the kernel binds in both arms. Which engine binds must be read from the s4 columns.

FP32 halves pedestal traffic: −41 % kernel time

- **~80 % of pixels** take the **pedestal-update** branch: six accumulator values read and written.
- One typedef halves all four pedestal arrays (**48 bytes per updating pixel in FP64, 24 in FP32**), and the kernel is **bandwidth-bound**.



Two more effects, neither the reason it works: FP64 arithmetic runs at 1/64 of FP32 on a GeForce part, and the narrower accumulators free 9 registers at 3×3.

ONE TYPEDEF

```
// clusterfinder_kernel.cuh
using COMPUTE_TYPE    = float;
using DEVICE_PED_TYPE = float;
//            was: double
```

Kernel, 9×9 **[s1 · cap 1700]**
39.86 → 23.70 µs (−40.5 %)

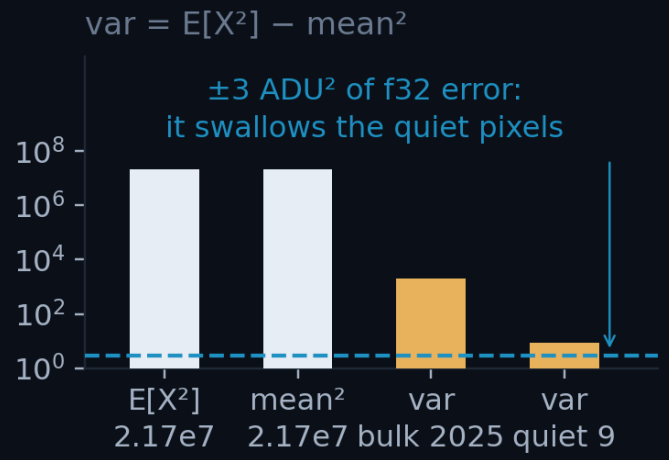
At 3×3 the same typedef is worth **−70.6 %** in the kernel and **4.6 %** end to end: there the kernel was never the tallest bar.

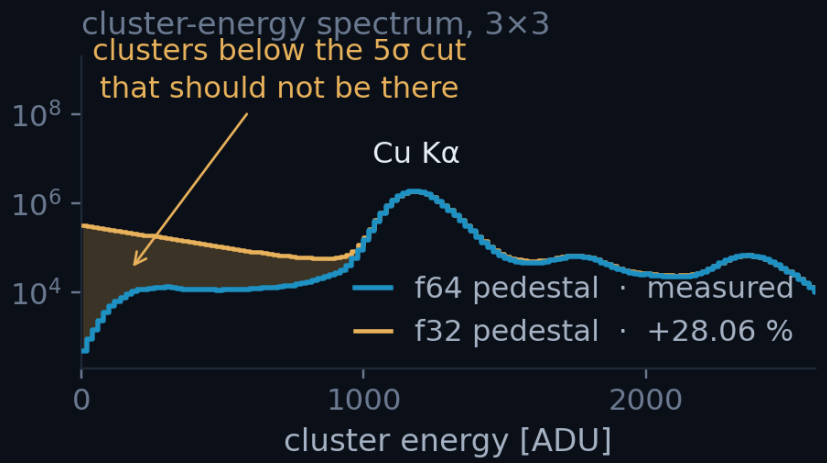
Naive FP32 is **wrong**. See **annex A9**.

Both builds, same git rev, 20 000 frames. Full engine grid: A7.

Accumulate what is small, not what is large

- **The trap. var = E[X²] − mean²** takes two numbers near 2 × 10^7 to get one near 2 000.
- **What it cost.** The rms **clamps to zero**, so a 5σ gate becomes a **0σ gate**.
- **The fix.** Accumulate the **centred Y = X − X_0**: both operands become O(rms).





Left: the operands and the answer against the ±3 ADU² error floor. Right: the f32 curve is reconstructed: measured area, modelled shape. The two-line patch is on the next slide.

NAIVE F32 · WHAT IT DID

EXTRA CLUSTERS

+28.06 %

PIXELS AFFECTED

~1–2 % of the sensor

ERROR FLOOR

±3 ADU², absolute

AFTER THE REWRITE

F32 VS F64 COUNTS

3 × 10^{-7}

VS THE CPU BASELINE

0.0039 %

The rewrite in full, and which pixels the error reached

- Freeze **X_0 = round(mean)** once, at the end of pedestal training, and never move it.
- Accumulate the **centred** $Y = X - X_0$; report **X_0 + sum/n**. Both operands are now **O(rms)-sized**.

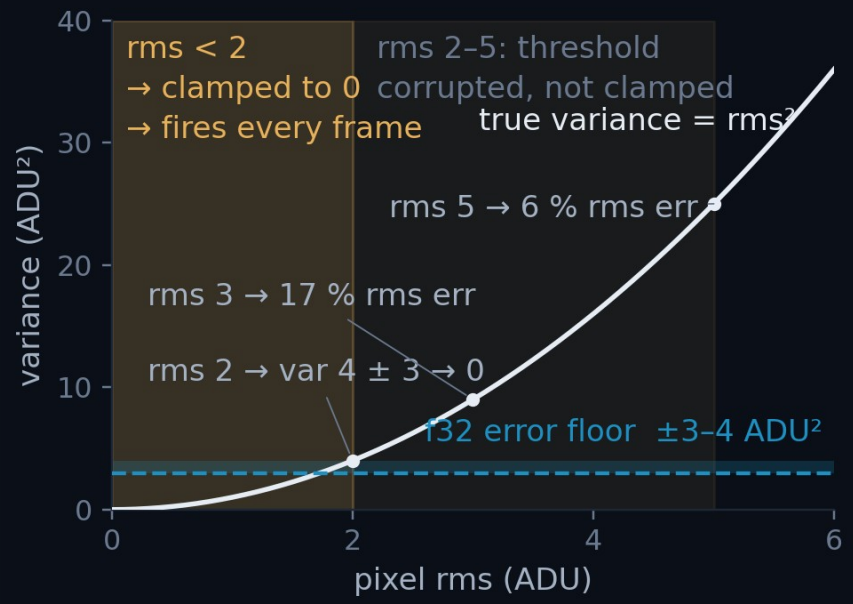
clusterfinder_kernel.cuh

```
// before: both terms ~2.17e7, answer ~2000
var = sum2/n - mean*mean;

// after: centred on a frozen per-pixel offset X0
DEVICE_PED_TYPE resid  = mean - d_pd_off[i];      // ~O(1)
DEVICE_PED_TYPE var_px = sum2[i]/n - resid*resid;  // no cancellation
```

Result: the 100 % FP32 build matches the FP64 build to **3×10^{-7}**, 70 clusters out of 233 million.

X_0 must never be updated: the accumulators are defined relative to it. Welford's is the other correct answer; this one is two lines and costs nothing.



Which pixels the ±3 ADU² floor actually reaches: the quiet ones, whose true variance is smallest.

The fault model, tested against every step

- The **0.68 µs per fault** rate is fitted on the **3×3 f32** ladder. Every row below is **9×9, f64 vs f32**, so the rate is applied **out of sample**, never refitted.

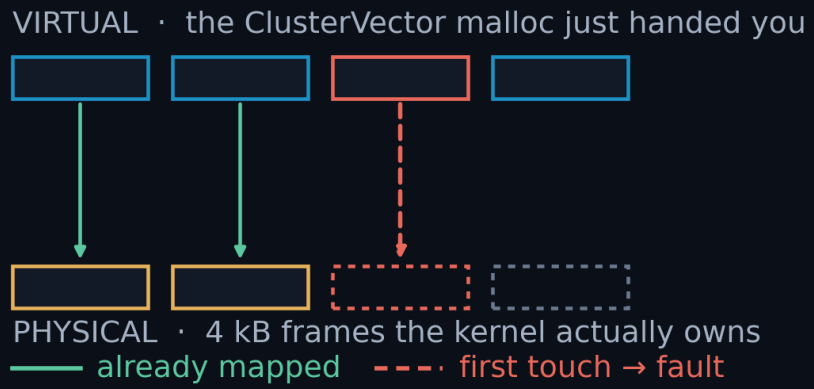
STEP	F64 WARM (FAULTS)	F32 WARM (FAULTS)	Δ WALL	Δ FAULTS	PREDICTED	VERDICT
opt3	82.44 µs (128 k)	95.66 µs (521 k)	**+13.22**	+393 k	**+13.37**	**the allocator**
opt4	79.83 µs (128 k)	75.20 µs (127 k)	−4.63	−0.5 k	−0.02	clean
opt5	66.39 µs (152 k)	61.85 µs (10 k)	−4.54	−141 k	**−4.81**	**not separable**
opt6	30.01 µs (0)	25.14 µs (0)	−4.87	0	0.00	clean

> **opt3 is the whole argument in one row.** A 393 k fault gap predicts **+13.37 µs**; **+13.22** was observed. The −40 % kernel is in there, invisible under 13 µs of the OS zeroing pages.

9×9 · cap 1 700 · warm = best of reps 1–4. The 0.68 µs/fault rate is carried in unchanged from the 3×3 fit: nothing here is tuned to make the columns agree.

Materialising clusters costs 2.6 M page faults

- Every run that keeps its results **allocates a fresh heap and touches it once**. Linux only finds physical memory on **first touch**, so that pass is where the OS works, **inside the timer**.



MINOR FAULT · in the kernel

1. traps into the kernel
2. finds a free frame
3. **zeroes all 4 kB of it**
4. maps it; the write proceeds

no disk: that would be a MAJOR fault

Re-run until minor faults plateau (< 200 k), and quote that run.

ONE 100 000-FRAME PASS · 3×3

PAGES FAULTED IN

~2.6 M

COST INSIDE THE TIMER

up to 4 s

CLUSTERS MATERIALISED

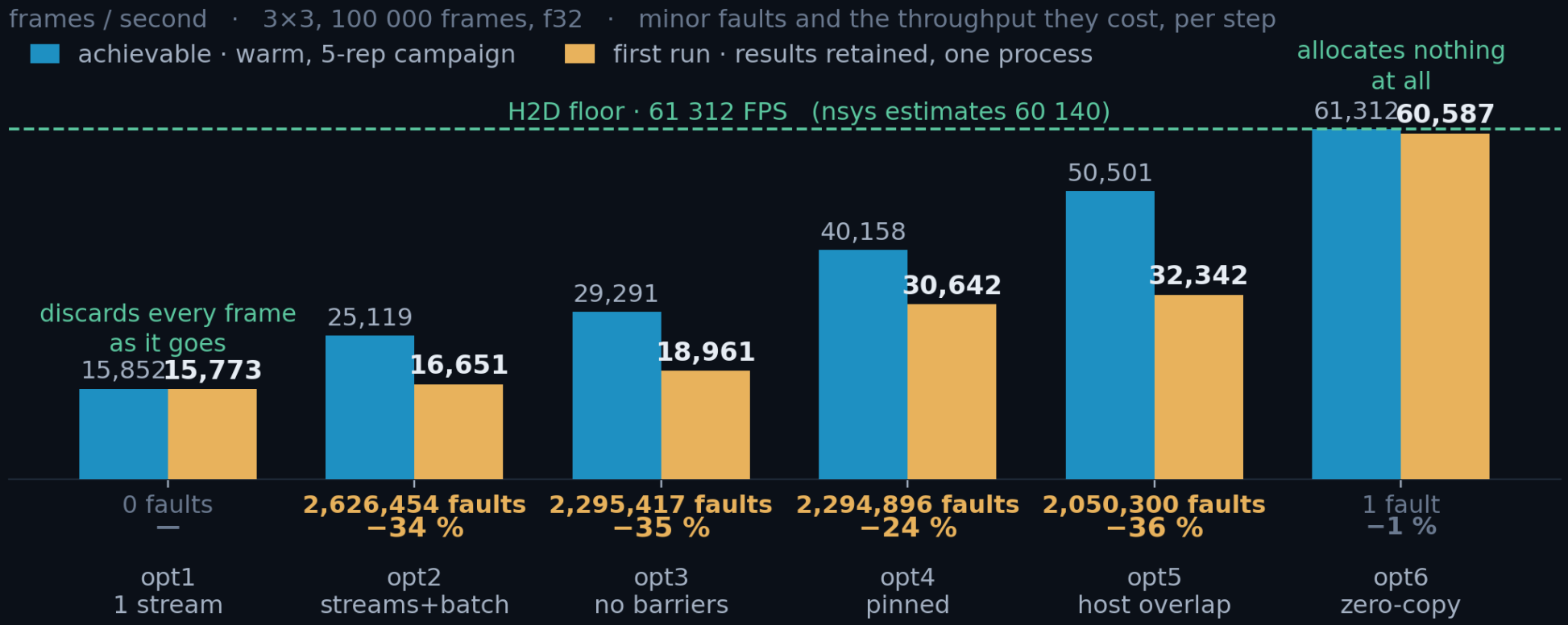
~10 GB

COST PER FAULT, FITTED

0.68 µs

Zero major faults all campaign. Copying faster cannot avoid this; not allocating can. That is opt6.

A first run loses a third of its throughput to page faults



Everything that materialises clusters loses a third of its throughput on the first run: +7 to +20 µs per frame.

Only the two ends escape, for opposite reasons. opt1 discards each frame; opt6 never grows a heap, and reaches **98 % of its peak on a cold process**.

Single pass, one process, every ClusterVector retained · f32 · 100 000 frames. Repeat the run and the amber bars climb onto the blue ones.

Three ways a GPU benchmark lies

First-touch page faults

Each run materialises ~10 GB of clusters. The first pass faults in ~2.6 M pages at 0.7 µs each, up to 4 s of pure OS work inside the timer.

Fix: re-run until getrusage() minor faults plateau (< 200 k).

CUDA-event kernel timing

avg_kernel_time_ms() measures elapsed time on a stream, including waiting for other streams. Under 8-stream load it over-reads by up to 3.5×.

Fix: Nsight Systems per-instance times; 1 stream for exclusive numbers.

The profiler itself

Under nsys, wall time per frame inflates ~4× from API tracing.

Fix: GPU op times from nsys, wall times from unprofiled runs.

THE FAULT PROTOCOL · BRACKET EVERY TIMED CELL

```
# bracket every timed cell:
mf0 = resource.getrusage(resource.RUSAGE_SELF).ru_minflt
...   t = time.perf_counter() - t0   ...
print(f'minor faults: {mf1-mf0:,}')   # quote the run where this plateaus
```

Validated: **wall = steady-state + faults × 0.68 µs** reproduced a 6.110 s run to within **1 ms**. Kernel time stayed constant throughout; the GPU was never the variable.

First-touch page faults: two sources, one counter

- A page exists in the address space but has no physical frame yet. First touch makes the kernel find one, **zero it** (mandatory), and map it. No disk: ru_majflt stays 0 all campaign. At 4 kB/page, **1 GB = 262 144 faults**.

	(A) RESULT HEAP	(B) PINNED D2H SLOTS
allocator	malloc → mmap, one ClusterVector per frame	cudaMallocHost, in submit_batch
cost / page	**0.7 µs**	**1.0 µs**: same fault + pin + DMA map
recurs?	**yes**, every alloc/free cycle above the mmap threshold	**no**: once per buffer, for its lifetime
removed by	**collect_view()**: it allocates nothing	nothing; reserve_output_slots() only moves it out of the timer

The two are exactly additive. Reserving subtracts precisely the pre-pin count from run 0 and changes nothing else.

NOT A CORRELATION, AN IDENTITY

```
3x3:   572 292 - 455 129 = 117 163   vs 117 192 pre-pin
9x9: 2 759 037 - 2 278 567 = 480 470   vs 480 474
closed form: 2 slots x 2000 x 120 004 B / 4 kB = 117 191
```

At 9×9 the heap never plateaus. ~9.3 GB per pass exceeds glibc's mmap threshold, so it is re-faulted every pass: **~10 µs/frame, permanently**.

CUDA events measure the stream, not the kernel

- avg_kernel_time_ms() brackets the launch with **cudaEventRecord on the kernel's own stream**, so it returns time on **that stream's timeline**, including time queued behind other streams.
- Honest at 1 stream, inflated up to **3.5×** at 8. The tell: *PCIe + overhead* = wall/N − kernel_ms **goes negative**.
- It is not free either: **~3.6 µs/frame**, 10–15 % of throughput at 3×3.

ClusterFinderCUDA.hpp

```
if (m_time_kernels)                        // OFF by default, all 3 finders
    cudaEventRecord(start[slot][i], sc.stream);
find_clusters_in_single_frame<<<grid, block, shmem, sc.stream>>>(...);
if (m_time_kernels)
    cudaEventRecord(stop[slot][i], sc.stream);   // <- queue-wait lands here
```

> The flag exists for **comparability**, not preference: with events on for one finder and off for another, the step between them absorbs the tax.

9×9 KERNEL · F64 · NSYS

S1 · DURATION

39.86 µs

S4 · OCCUPANCY

32.66 µs

OVERLAP FACTOR

1.32×

Same kernel. The s4 column is the union of kernel intervals per frame, not how long one kernel takes. Quote s1 for duration; s4 feeds the floor, subject to the sustained-rate rule in A7, which has the full grid.

Where nsys is sound, and where it is not

- Tracing a **host** call runs a callback on entry and exit, **inside the interval being measured**. GPU work is stamped by the hardware and read back **afterwards**, so nothing is injected into the execution path.

MEASUREMENT	SQLITE TABLE	9×9 [F64 · S1 · CAP 1500]	VERDICT
cudaLaunchKernel: the host call	CUPTI_..._RUNTIME	1.85 µs	**inflated ~4×**
the kernel executing	CUPTI_..._KERNEL	39.93 µs	sound to ~2 %
cudaMemcpyAsync: the host call	CUPTI_..._RUNTIME	1.65 µs	**inflated ~4×**
the H2D / D2H transfer	CUPTI_..._MEMCPY	13.25 / 19.44 µs	sound to ~2 %

> **Same cudaMemcpyAsync, two numbers: 1.65 µs to ask for the copy, 13.25 µs for the copy to happen.** A profiler that must be present to measure,distorts; one that reads a record later does not.

Every engine number in this deck comes from GPU-side timestamps. Proof they are sound: opt7 sustains 25.14 µs unprofiled against a 25.24 µsprofiled estimate, 0.4 % apart.